Applied Soft Computing Journal 201 (2026) 115710

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Chance constraint-based dependent task offloading for multi-access edge computing under uncertainty

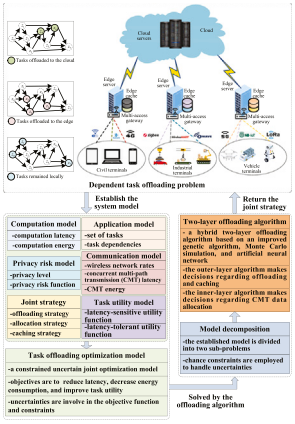
Xiaomin Jin [a, b, c, *], Jingbo Wang [a, b, c], Zhongmin Wang [a, b, c], Yanping Chen [a, b, c]

[a] School of Computer Science and Technology, Xi'an University of Posts and Telecommunications, Chang'an West Street, Xi'an, 710121, Shaanxi, China
[b] Shaanxi Key Laboratory of Network Data Analysis and Intelligent Processing, Chang'an West Street, Xi'an, 710121, Shaanxi, China
[c] Xi'an Key Laboratory of Big Data and Intelligent Computing, Chang'an West Street, Xi'an, 710121, Shaanxi, China

HIGHLIGHTS

- We establish an uncertain joint optimization model for dependent task offloading in multi-access edge computing and prove its NP-hardness.
- We decompose the uncertain optimization model into two chance-constrained sub-problems and propose a hybrid two-layer offloading algorithm.
- We enhance task offloading through the combined utilization of concurrent multi-path transmission, edge caching, and privacy protection.

GRAPHICAL ABSTRACT



ARTICLE INFO

Keywords:
Multi-access edge computing
Cloud-edge-end collaboration
Task offloading
Task dependency
Uncertainty

ABSTRACT

As a novel computing paradigm, multi-access edge computing (MEC) empowers resource-constrained mobile terminals with robust and versatile computational capabilities. However, the development of precise offloading strategies, which determine the execution location for tasks, remains a prominent and unresolved challenge for MEC. The real-life MEC environment is characterized by its non-ideal and uncertain nature. In progressively sophisticated applications, intricate dependency relationships among their tasks continuously intensify. These factors significantly amplify the challenges associated with developing efficient offloading strategies. In this paper, we study the dependent task offloading problem for MEC under uncertainty with joint consideration of concurrent multi-path transmission, task caching, and privacy protection. Firstly, we employ stochastic numbers to effectively characterize uncertainties and formulate the task offloading problem as a constrained stochastic joint optimization model. We establish an uncertain joint optimization model to formulate the problem of dependent task offloading in MEC under uncertainty and prove its NP-hardness. The established model focuses on optimizing the joint strategy to achieve the objectives of reducing latency, decreasing energy consumption, and enhancing task utility. Secondly, we decompose the model into two chance-constrained sub-problems and propose a hybrid two-layer offloading algorithm that combines an improved genetic algorithm, Monte Carlo simulation,

* Corresponding author at: School of Computer Science and Technology, Xi'an University of Posts and Telecommunications, Chang'an West Street, Xi'an, 710121, Shaanxi, China.
Email address: xmjin@xupt.edu.cn (X. Jin).

https://doi.org/10.1016/j.asoc.2026.115710
Received 25 August 2024; Received in revised form 17 April 2026; Accepted 3 June 2026
Available online 10 June 2026
1568-4946/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

and artificial neural networks to solve the joint strategy. The proposed algorithm leverages the collaborative synergy between inner and outer layers to solve the joint strategy. The experimental results demonstrate that the established model exhibits significant effectiveness in reducing the objective value, achieving a 34.53% decrease in the objective value compared to local execution. Our offloading algorithm surpasses existing algorithms and attains an average reduction of at least 20.34% in the objective value.

1. Introduction

In recent years, with the rapid development of the Internet of Things (IoT), mobile terminals have been integrated into all aspects of people's daily lives, which has inspired many novel applications, such as fingerprint and facial recognition, virtual reality, and augmented reality [1,2]. Tasks generated by these applications are often computationally intensive and latency-sensitive, placing higher demands on terminals' computational capacities. However, there are limitations on the computational and energy resources of mobile terminals mandated by mobility and portability requirements, resulting in some tasks that cannot be executed locally and often need to be offloaded to an external platform for execution. Unfortunately, as a traditional external platform, the long backhaul of cloud computing presents several challenges in real-time computational resources, data transmission security, and communication reliability [3]. To address the limitations of cloud computing, multi-access edge computing (MEC) adds edge servers positioned between the cloud and mobile terminals. This architectural enhancement aims to relocate computational resources toward the network edge, thereby reducing proximity to mobile users and facilitating efficient computational execution [4]. MEC provides a complete "cloud-edge-end" system architecture, which enables the option to either keep tasks local, offload them to the edge, or further offload them to the cloud. As the hierarchy progresses from the mobile terminal to the cloud, computational resources become abundant, while simultaneously, there is a corresponding increase in transmission latency. Through the collaboration of the cloud, edge, and end (mobile terminal), MEC offers a compelling solution by delivering potent and adaptable computing power. Consequently, MEC has the ability to effectively meet the distinct demands of various task types in terms of latency and computational resources [5].

The offloading strategy plays a pivotal role in determining the execution location for tasks and significantly influences the MEC performance. An inaccurate offloading strategy will result in many additional data transfers, thus triggering greater consumption than local execution. Also, the MEC performance is significantly influenced by wireless network conditions, leading to a direct impact on the development of offloading strategies. Real-life wireless networks are characterized by inherent uncertainty and constant fluctuations due to various factors such as multi-path propagation, the Doppler effect, the shadowing effect, and interference. The dynamic nature of wireless networks poses considerable challenges in accurately adapting to their evolving state, thereby necessitating robust offloading strategies that can effectively handle the uncertainties in wireless network conditions.

The wireless network serves as the vital "pipeline" that establishes the connection between mobile terminals and the edge/cloud during task offloading. The reliance on wireless networks introduces a prominent bottleneck that hinders offloading performance. At the same time, real-life environments encompass multiple wireless networks. Smartphones, as the most representative mobile terminals, are equipped with diverse network interfaces, including but not limited to 4G/5G, WiFi, and Bluetooth. Several emerging wireless transmission technologies also facilitate data transmission across multiple channels. If multiple wireless networks or channels are utilized simultaneously for data transmission, it has the potential to increase the data transmission rate and enhance the robustness of the MEC system. The availability of mature concurrent multi-path transmission (CMT) [6] technologies, such as multi-path transmission control protocol (MPTCP), stream control transmission protocol (SCTP), orthogonal frequency division multiple access (OFDMA), and non-orthogonal multiple access (NOMA), has established a realistic foundation for integrating CMT into task offloading. The edge, located just one hop away from mobile terminals, serves as an efficient caching platform. Leveraging caching at the edge facilitates the avoidance of redundant computations in tasks, thereby enhancing offloading performance and optimizing the utilization of MEC resources. Mobile terminals store a vast amount of personal information, including biological and property-related data. This abundance of sensitive data poses a significant threat to privacy when it comes to task offloading. The transfer of data across different levels of "cloud-edge-end" entails the inherent risk of unauthorized collection, storage, analysis, and distribution of information. Edge and cloud servers are also susceptible to a wide range of cyber-attacks, which further emphasizes the urgency of addressing the issue of safeguarding privacy during task offloading [7,8].

Nonetheless, a significant limitation in most existing work is the prevalent assumption of an ideal and deterministic MEC environment during task offloading. For example, the uncertain nature of key factors such as uplink and downlink bandwidths is overlooked in [9,10] when tasks are offloaded to the edge or cloud. Such a simplified perspective causes the developed offloading strategy to fail to account for the inherent uncertainty and dynamic nature of real-world MEC environments. In addition, with the increasing complexity of applications, the internal structure of applications has gradually evolved into a composition of multiple dependent tasks. The dependencies among tasks further exacerbate the complexity and challenges associated with task offloading decisions. When developing offloading strategies, it is essential to comprehensively consider the dependencies among tasks and the data interactions among them [11]. Besides the aforementioned uncertainties and task dependencies, CMT, task caching, and privacy protection emerge as significant considerations that demand careful attention in the MEC task offloading.

The primary motivation of this paper is to develop a comprehensive approach for addressing the dependent MEC task offloading problem under uncertainty with the joint consideration of CMT, task caching, and privacy protection. We propose an innovative approach that includes a novel uncertain task offloading model and a novel hybrid two-layer offloading algorithm. Firstly, we establish an uncertain joint optimization model for dependent task offloading in MEC, aiming to reduce latency, decrease the energy consumption, and enhance task utility. In the established model, stochastic numbers are employed to characterize uncertainties, while a dependent application model is utilized to capture the dependency relationships and data interactions among its tasks. The established model incorporates CMT for efficient data transfer among tasks, as well as task caching on the edge. The strategy in this model is comprehensive, encompassing the task offloading strategy, CMT task data allocation strategy, and task caching strategy, all of which are jointly optimized. Furthermore, the established model incorporates a privacy risk model for the task and utilizes it as a constraint to ensure the privacy security of task offloading. We prove that the established model is NP-hard. Secondly, to develop the joint strategy from the NP-hard model, we propose a hybrid two-layer offloading algorithm based on an improved genetic algorithm, Monte Carlo simulation, and artificial neural networks. The proposed offloading algorithm decomposes the uncertain joint optimization model into two chance constrain sub-problems. Based on the problem decomposition, the outer-layer algorithm is designed to address the task offloading strategy and edge

caching strategy, while the inner-layer algorithm is designed to address the CMT task data allocation strategy. The genetic algorithm is improved in terms of initialization and genetic operations to effectively explore optimal strategies. Monte Carlo simulation is employed to calculate the chance constraint values. However, the direct utilization of Monte Carlo simulation suffers from the drawback of redundant calculations. To address this issue, we integrate Monte Carlo simulation with artificial neural networks, enabling the offloading algorithm to learn and predict chance constraint values. This improvement and integration enable the offloading algorithm to address chance constraints with increased efficiency and optimize the search for optimal strategies. Finally, we conduct a series of experiments to thoroughly validate and evaluate both the established model and the proposed algorithm. The main contributions of this paper can be summarized as follows:

1. We establish an uncertain joint optimization model to formulate the problem of dependent task offloading in MEC under uncertainty and prove its NP-hardness. We incorporate stochastic numbers to effectively characterize uncertainties and employ a dependent application model to capture task dependencies and data interactions among its tasks. Furthermore, we integrate CMT, task caching, and privacy protection into the model. Our model focuses on jointly optimizing the task offloading strategy, CMT task data allocation strategy, and task caching strategy to achieve the objectives of reducing latency, decreasing energy consumption, and enhancing task utility.
2. We propose a hybrid two-layer offloading algorithm that combines an improved genetic algorithm, Monte Carlo simulation, and artificial neural networks to effectively solve the established uncertain joint optimization model. We decompose the model into two chance-constrained sub-problems and design an offloading algorithm that consists of an outer layer and an inner layer. A collaborative utilization of inner and outer layers is employed to effectively address the joint strategy. To efficiently handle chance constraints, integration of Monte Carlo simulation with the artificial neural networks is employed, while simultaneously improving the genetic algorithm in terms of initialization and genetic operations to effectively explore the optimal strategies.
3. We validate the established system model and evaluate the proposed offloading algorithm through extensive simulation experiments. Our experiments thoroughly examine the impact of various system parameters on the optimization objective. To assess the efficiency of our offloading algorithm, we compare it against five existing algorithms.

The remainder of this paper is organized as follows. Section 2 reviews the related work. Section 3 introduces the system model. The offloading algorithm is illustrated in Section 4. In Section 5, the established system model and proposed offloading algorithm are evaluated. Section 6 concludes this paper.

2. Related work

2.1. Task offloading under uncertainty

The majority of existing work assumed an ideal MEC environment and established deterministic task offloading models using classical optimization theory. Subsequently, task offloading algorithms were designed based on heuristic algorithms. However, it is important to acknowledge that real-life MEC environments often deviate from these ideal assumptions, necessitating further research to address the challenges posed by uncertainties. Some researchers have identified this issue and conducted research to tackle the challenge of task offloading under uncertainty.

Zhang et al. studied the task offloading problem in vehicular edge computing under uncertainty and formulated it as a finite horizon Markov decision process to minimize the latency [12]. They solved the model by employing backward dynamic programming through the Bellman optimality equation. Zhou et al. studied the task offloading problem for edge computing under information uncertainty and proposed a matching-learning-based online offloading algorithm by combining the pricing-based matching [13]. Sun et al. studied the dynamic task offloading problem in vehicular edge computing and proposed an adaptive learning-based task offloading algorithm based on the multi-armed bandit theory [14]. Li et al. studied the robust task offloading problem in the multi-unmanned aerial vehicle-assisted MEC network and formulated it as a multi-agent Markov decision process [15]. They proposed a multi-agent proximal offloading algorithm with a Beta distribution framework to achieve a flexible offloading strategy. Zhu et al. investigated the problem of wireless charging and partial task offloading in time-varying MEC, taking into account social relationships and the residual energy of terminals [16]. They formulated the optimization problem as a Markov decision process and proposed a deep reinforcement learning approach based on hybrid actor–critic networks. Wang et al. developed a multi-user MEC task offloading framework that considers unknown system information and formulated the dynamic task offloading problem as an online multi-user multi-armed bandit process [17]. They divided the time horizon into epochs and proposed a decentralized epoch-based offloading algorithm. Wu et al. studied the dependency-aware task offloading problem in the Internet of Vehicles and proposed a heterogeneous graph attention network augmented deep reinforcement learning-based approach [18]. Khoramnejad et al. studied the dependent task offloading problem in unmanned aerial vehicles-assisted MEC and formulated it as a long-term optimization model [19]. In addition to the uncertainty in MEC environments, some work also investigated the task offloading problem under the uncertainty of the number of CPU cycles required for task execution. Li et al. considered the uncertainty in the number of required CPU cycles of a task and aimed to minimize the average long-term energy consumption per task [20]. They employed an exact conic reformulation to convert probabilistic constraints into deterministic constraints for the development of the offloading strategy. Chang et al. also took into account the uncertainty in the number of CPU cycles required for task execution and formulated a model aiming to minimize the average response latency under the energy consumption constraint [21]. They proposed an offloading algorithm based on the Lagrange multiplier to obtain the optimal offloading strategy. Eshraghi and Liang considered the uncertainty in task computing requirements and formulated the task offloading problem as a mixed-integer program model [22]. To address the uncertainty in the model, they employed expected values and transformed it into a convex programming problem that was subsequently solved.

2.2. Joint task caching with task offloading

Edge task caching offers several advantages, including reduced latency and alleviation of redundant computations, achieved by storing previously executed tasks at the edge. Edge task caches check for the availability of cached tasks when a new task request is received. If a cached version of the task is found, it can be directly retrieved and executed, eliminating the need for repetitive computations. Some researchers have conducted studies on the task offloading problem by investigating the integration of edge task caching.

Peng et al. utilized an open Jackson queuing network to formulate the task offloading problem with the joint consideration of task caching [23]. They optimized the long-term problem on a slot-by-slot basis and employed the Lyapunov drift-plus-penalty function to solve the problem. Liu et al. studied the collaborative task offloading problem with the aid of dynamic task caching and formulated it as a mixed optimization model [24]. They proposed a two-level computing strategy to solve the optimized problem. Shen et al. formulated the problem of task offloading and caching as a mixed-integer non-linear programming model [25]. They developed a dynamic programming-based semi-distributed algorithm to solve the optimization model. Feng

et al. formulated the joint task offloading problem as a mixed-integer nonlinear programming model and subsequently transformed it into an equivalent convex optimization model [26]. The transformed model has two equivalent sub-problems of task offloading and capacity-constrained caching knapsack. They proposed a distributed algorithm to optimize the data caching strategy by dynamically selecting the cached content and assigning it to different edge servers for collaborative caching. Xiao et al. studied the joint task caching and offloading problem in device to device-aided MEC and formulated it as a two-objective optimization model [27]. They proposed an enhanced binary particle swarm optimization algorithm and an improved two-objective bat algorithm to solve their model. Ma et al. proposed an online algorithm that effectively caches services and manages service states in an online manner [28]. Their approach leveraged the Lyapunov method to transform the initial problem into a sequence of single-time frame minimization problems. Zhang et al. studied the offloading problem of inter-application dependent tasks and proposed an offloading strategy under the condition of limited computing and cache resources [29]. Zhao et al. proposed a dependency-aware dynamic caching framework for task offloading, incorporating an adaptive caching strategy designed to achieve load balancing [30]. Zhang and Yu proposed a hybrid caching strategy combining popular and coded caching to resolve the conflict between limited edge caching and surging data demand [31]. They used a two-step adaptive genetic algorithm to jointly optimize cache placement and task offloading.

2.3. Joint CMT with task offloading

CMT facilitates concurrent transmission of task data across multiple wireless networks or channels, effectively addressing the performance constraints imposed by wireless networks on MEC task offloading. By concurrently utilizing multiple wireless networks or channels, CMT demonstrates a substantial enhancement in the overall throughput of task offloading. Moreover, CMT effectively mitigates the reliance on a singular network or channel, thus providing greater resilience to network failures and variations in signal quality during the task offloading process. Some researchers have integrated CMT with task offloading to augment MEC performance.

Song et al. utilized the benefits of NOMA to enhance transmission efficiency and aimed to reduce users' energy consumption effectively in MEC through the utilization of NOMA [32]. The problem was formulated as a constrained optimization model, and a joint algorithm for both single-user and multi-user scenarios was proposed. Jin et al. studied the task offloading problem and explored the joint utilization of multiple wireless networks with the objective of minimizing both latency and energy consumption [33]. They established a dynamic task offloading model and proposed a runtime algorithm to solve the online offloading strategy. Wang et al. investigated the task offloading problem by incorporating multi-antenna NOMA to enhance the performance of multi-user MEC systems and formulated it as a convex optimization model [34]. Li et al. conducted a study on the joint sub-carrier and power allocation in MEC systems based on OFDMA, considering partial task offloading [35]. In this scenario, a portion of the data can be computed locally on the mobile terminal, while the remaining portion is transmitted to the edge for computing. They proposed a heuristic algorithm that separates sub-carrier assignment and power allocation to solve their optimization model. Nath et al. investigated the task offloading and resource allocation problem in a multi-user multi-channel MEC environment [36]. They assumed the presence of multiple wireless channels, where each task has the flexibility to traverse any of these channels. They employed a deep reinforcement learning-based approach to solve their problem. Cao et al. studied the multi-channel access and task offloading problem in MEC-enabled Industry 4.0 and proposed a multi-agent deep deterministic strategy gradients-based algorithm [37]. Xue and An studied the MEC task offloading problem with the joint utilization of NOMA and formulated it as a mixed integer nonlinear programming model with the goal of maximizing the system's processing capability [38]. They decomposed the model into two sub-problems and employed their proposed low-complexity sub-optimal matching algorithm and the Lagrangian multiplier method to solve these sub-problems. Yu et al. conducted a study on the task offloading problem, focusing on the joint utilization of OFDMA to achieve energy savings in MEC [39]. They proposed a joint offloading algorithm that optimizes both wireless and computational resources using OFDMA.

2.4. Joint privacy protection with task offloading

Task offloading encompasses the transfer of data from mobile terminals to the edge or cloud. Mobile terminals often contain a plethora of personal, sensitive, or confidential information. Consequently, privacy protection plays a vital role in maintaining the security and confidentiality of task offloading processes, thereby preventing unauthorized access or disclosure of private data. By acknowledging the importance of privacy preservation, some researchers have striven to study the task offloading problem in relation to the protection of privacy.

Wang et al. established an optimization model for task offloading that incorporates joint data compression and privacy protection [40]. They proposed an offloading algorithm based on the improved simulated annealing particle swarm algorithm to solve the offloading strategy. Samy et al. introduced a blockchain-based framework for secure task offloading in MEC with the primary objectives of minimizing latency and energy consumption [41]. They formulated the task offloading problem as an NP-hard model and proposed a deep reinforcement learning-based offloading algorithm to derive a close-to-optimal offloading strategy. Luo et al. presented an architecture for automatically distributed offloading systems in drone-based MEC and designed a smart contract to integrate with offloading strategies [42]. Their private blockchain established a permissioned network wherein only specific members are permitted to participate, enabling the exclusion of untrustworthy MEC service providers. He et al. proposed an architecture for a MEC-enabled unmanned aerial vehicle-assisted ad hoc network [43]. Their study focused on the task offloading problem by simultaneously considering the transmission and security assurance models. They proposed an iterative algorithm based on the relax-and-rounding and Lagrangian methods to solve the joint optimization problem. Xu et al. presented a two-phase task offloading optimization strategy aiming to optimize both the offloading utility and privacy in the context of edge computing-enabled IoT [44]. They devised a joint optimization approach to balance privacy preservation and execution performance. Sun et al. studied the problem of privacy-preserved secure task offloading [45]. Their scheme focused on minimizing energy consumption while considering the preservation of location privacy, usage pattern privacy, and secure transmission against potential eavesdroppers. They formulated this problem as a constrained Markov decision process and solved it with reinforcement learning. Fan et al. presented a collaborative-filtering privacy-preserving vehicular edge computation offloading approach, which provides high security and protects computational resource privacy [46]. Zhu et al. investigated the trust-aware task offloading problem with cache constraints in MEC networks, jointly considering social relationships, task offloading, and caching [47]. The Kuhn–Munkres algorithm was employed to facilitate optimal matching between tasks and terminals, thereby enabling the selection of an appropriate task offloading strategy.

2.5. Brief summary

Table 1 presents a comprehensive comparison of the related work. The majority of existing research has primarily focused on deterministic task offloading problems, which assume ideal environments and constant system parameters. These studies typically employed classical optimization theory to formulate deterministic optimization models and subsequently proposed offloading algorithms based on heuristic approaches for model solutions. Some researchers have also started considering the problem of uncertain task offloading, primarily addressing it through

Table 1
Comparison of the related work.

Work	Architecture	Uncertainty considered	Dependency	Task caching	CMT	Privacy protection	Objective
[12]	edge-end	✓	Linear dependency	×	×	×	Minimize the latency
[13]	edge-end	✓	×	×	×	×	Minimize the latency
[14]	edge-end	✓	×	×	×	×	Minimize the latency
[15]	edge-end	✓	×	×	×	×	Minimize the energy consumption
[16]	edge-end-end	✓	×	×	×	✓	Maximize the social-energy utility
[17]	edge-end	✓	×	×	×	×	Maximize the user reward
[18]	cloud-edge-end	✓	DAG	×	×	×	Minimize the latency and energy consumption
[19]	edge-end	✓	DAG	×	×	×	Minimize the latency and energy consumption
[20]	edge-end	✓	×	×	×	×	Minimize the energy consumption
[21]	edge-end	✓	×	×	×	×	Minimize the latency
[22]	cloud-edge-end	✓	×	×	×	×	Minimize the weighted sum of expected cost and cost variation
[23]	cloud-edge-end	✓	Linear dependency	✓	×	×	Minimize the latency
[24]	edge-end	×	×	✓	×	×	Minimize the latency
[25]	edge-end	×	DAG	✓	×	×	Minimize the latency and energy consumption
[26]	edge-end	×	×	✓	×	×	Minimize the latency and energy consumption
[27]	edge-end	×	Linear and parallel dependency	✓		×	Minimize the latency and energy consumption
[28]	edge-end	✓	DAG	✓	×	×	Minimize the system cost
[29]	cloud-edge-end	×	DAG	✓	×	×	Minimize the latency and energy consumption
[30]	edge-end	✓	DAG	✓	×	×	Minimize the average latency
[31]	cloud-edge-end	×	×	✓	×	×	Minimize the average latency
[32]	edge-end	×	×	×	✓	×	Minimize the energy consumption
[33]	cloud-end	✓	DAG	×	✓	×	Minimize the latency and energy consumption
[34]	edge-end	×	×	×	✓	×	Minimize the energy consumption
[35]	edge-end	×	×	×	✓	×	Minimize the latency
[36]	edge-end	×	×	×	✓	×	Minimize the latency and energy consumption
[37]	cloud-edge-end	✓	×	×	✓	×	Reduce the latency and improve the channel access success rate
[38]	cloud-edge-end	×	×	×	✓	×	Maximize the system's task processing capacity
[39]	edge-end	×	×	×	✓	×	Minimize the energy consumption
[40]	edge-end	×	×	×	×	✓	Minimize the latency and security risk
[41]	cloud-edge-end	×	×	×	×	✓	Minimize the latency and energy consumption
[42]	edge-end	×	×	✓	×	✓	Reduce the latency and improve the visibility of MEC service providers
[43]	edge-end	×	×	×	×	✓	Minimize the latency
[44]	cloud-edge-end	×	×	×	×	✓	Maximize the resource utilization and minimize the latency
[45]	edge-end	✓	×	×	×	✓	Minimize the energy consumption
[46]	cloud-edge-end	×	DAG	×	×	✓	Minimize the latency
[47]	edge-end-end	×	×	✓	×	✓	Minimize the latency and energy consumption
Ours	cloud-edge-end	✓	DAG	✓	✓	✓	Minimize the latency, minimize the energy consumption, and maximize the task utility

the use of expectations or time slot segmentation. Expectation-based approaches represent uncertain variables using their expected values, while the resulting offloading strategies may exhibit limited adaptability to dynamic environments. Conversely, time slot segmentation-based approaches divide time into discrete slots, assuming fixed values for uncertain variables within each slot. This transformation enables the conversion of uncertain task offloading problems into multiple deterministic affiliated sub-problems. Most existing time slot segmentation-based approaches typically model the uncertain offloading optimization problem as a Markov decision process. They then usually learn an optimal strategy to maximize cumulative rewards by iteratively interacting with the uncertain environment through deep reinforcement learning (DRL) in an exploration–exploitation paradigm. The key advantage of DRL-based approaches is that they do not require an explicit analytical model of the system dynamics. However, DRL-based approaches exhibit several limitations when applied to the specific problem considered in this paper. Firstly, their reliance on trial-and-error mechanisms incurs substantial repetitive computations, leading to low computational efficiency when handling the coupled decision variables of offloading, caching, and CMT data allocation, which vastly expand the solution space. Secondly, for directed acyclic graph (DAG)-structured dependent tasks, the action space grows exponentially with task count and dependencies, leading to convergence challenges and often requiring specialized mechanisms such as action masking. Thirdly, they typically assume full observability and well-defined rewards, making it difficult to enforce chance constraints, as satisfaction cannot be strictly guaranteed even with reward shaping.

The application model in existing work predominantly follows a discrete paradigm, where an application contains only one task and tasks (applications) are considered to be mutually independent. This application model has a coarse granularity and does not consider the interactions among tasks. Some studies consider the dependencies among tasks, mainly in the form of linear dependencies and DAG dependencies. In the case of linear dependencies, tasks are executed in a specific order, where the output of the previous task serves as the input for the current task, and the tasks are executed linearly. On the other hand, DAG dependencies involve complex interactions among tasks, and there is no fixed execution order, representing a finer-grained application model.

CMT, task caching, and privacy protection are three crucial techniques for enhancing task offloading. These techniques can further improve network capabilities, eliminate redundant computations, and enhance privacy beyond basic task offloading. Existing work predominantly focuses on individual aspects, with few considering a combination of two of these techniques. To the best of our knowledge, there is currently no research that integrates all three techniques into task offloading.

Compared to existing work, this paper establishes a novel uncertain joint optimization model by formulating the problem of dependent

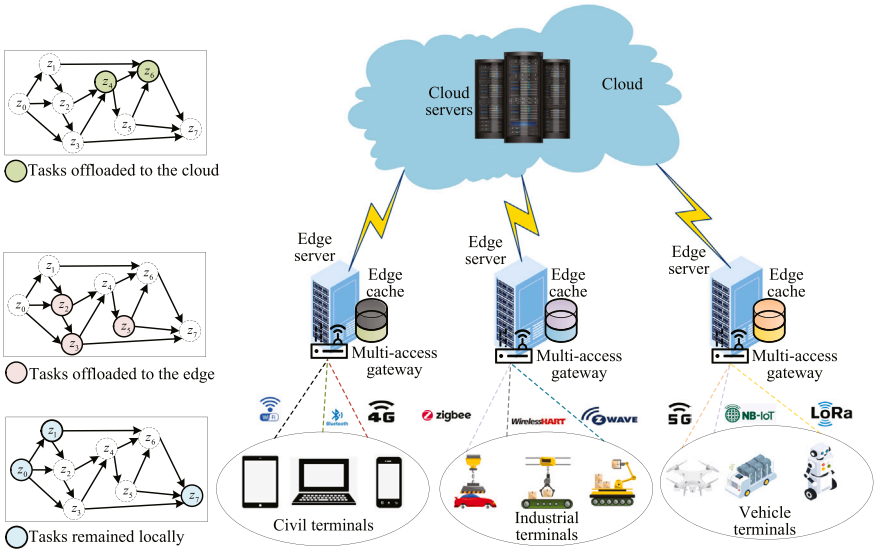


Fig. 1. The MEC system.

task offloading in MEC under uncertainty as a constrained stochastic optimization model, providing robust offloading strategies. This model concurrently incorporates CMT, task caching, and privacy protection to enhance the performance of task offloading. This paper also proposes a novel hybrid offloading algorithm that combines Monte Carlo simulation for accurate uncertainty characterization, neural network surrogates for efficient inference, and a two-layer hierarchical decomposition to separate caching/offloading decisions from CMT data allocation. Compared with DRL-based approaches, our approach addresses the challenges of chance constraints and task dependencies while maintaining computational tractability by avoiding redundant computations. Furthermore, it effectively overcomes the poor adaptability inherent in expectation-based approaches.

3. System model and problem formulation

3.1. System model

3.1.1. System architecture

The MEC system has the "cloud-edge-end" architecture illustrated in Fig. 1. The cloud establishes a reliable and efficient communication channel with the edge through high-speed fiber-optic links. Edge servers are deployed at the multi-access gateways and thus are connected to mobile terminals through wireless networks with one-hop distance. The edge provides an excellent platform for task caching, thereby eliminating the need for redundant executions when some tasks are offloaded to the edge. Considering the presence of multiple wireless network interfaces equipped with modern mobile terminals, we utilize CMT to facilitate simultaneous data transmission between mobile terminals and the edge across multiple wireless networks. As depicted in Fig. 1, vehicle terminals are equipped with three heterogeneous wireless interfaces: 5G, NB-IoT, and LoRa. The task data can be partitioned into three segments, with each segment transmitted simultaneously via a different interface. By distributing data across three interfaces, CMT not only increases the overall transmission throughput but also improves transmission reliability through path diversity. The applications originating from mobile terminals are characterized by fine granularity, comprising multiple tasks that exhibit DAG dependencies. These tasks offer the flexibility of being executed either locally or offloaded to the edge or cloud for execution. The tasks are categorized into three parts based on the offloading strategy: a portion is offloaded to the cloud for execution, another portion is offloaded to the edge for execution, and the remaining portion is executed locally. The presence of DAG dependencies among tasks signifies that the successful completion of a specific task is contingent upon the completion of other related tasks, thereby forming a chain of dependencies. Consequently, it is imperative to account for the execution sequence and data interactions among tasks when making offloading strategies, thus ensuring the accurate and orderly execution of tasks. The notations used in this paper are summarized in Table 2.

3.1.2. Application model

In order to accurately depict the interdependencies among tasks within an application, the DAG is employed as a representation of the application. We adopt a DAG model similar to that in [48], which originates from a real-world application and thus has strong practical grounding. A key feature of this application model lies in the input dependency, wherein each successor task receives at least a portion of its input from its previous task. As shown in Fig. 1, the output of one task may serve as the input for subsequent tasks, and the execution of a task is dependent on the completion of preceding pioneer tasks. All tasks are classified into three categories based on the offloading strategy, with subsequent execution distributed across three designated locations within the MEC system. The DAG application is a graphical structure comprising nodes and directed edges, wherein each node represents a task and each directed edge signifies a dependency between two tasks. The DAG application model can be formally defined as a graph $G = \{N, E\}$, where N represents the set of nodes and E represents the set of edges.

The set of tasks is denoted as $N = \{z_1, z_2 \ldots, z_n\}$, where z_i represents the ith task. A task is defined by four attributes, and a four-tuple $z_i = (d_i, c_i, \mu_i, \delta_i)$ $(1 \leq i \leq n)$ is employed to represent the ith task. In the attributes of task z_i, its input size is denoted by d_i, its workload is denoted by c_i, its output size is denoted by δ_i, and its privacy level is denoted by μ_i.

Edges within G represent the intercommunication among tasks, explicitly defining their dependencies. The edge $\tau_{i,j} \in E$ signifies that task z_i serves as a predecessor to task z_j, implying that task z_i must complete its execution prior to task z_j and transmit its output as input to task z_j. Data transfer between two tasks is solely necessary when they are offloaded to distinct locations. Consequently, when two tasks are offloaded to the same location, their communication cost is considered negligible.

3.1.3. Computation model

Mobile terminals can leverage resources from the edge or cloud to execute tasks through task offloading. However, task offloading incurs

Table 2
Notations.

Notation	Definition	Notation	Definition
f^l	The computational resources of the mobile terminal	$r^{up}_{i,w}$, r^{down}_i	The uplink and downlink rates for transferring task z_i's data on the wth wireless network
f^e_i	The computational resources allocated by the edge for the execution of task z_i	$t^{up}_{i,tran}$	The transmission latency for uploading task z_i's data form the mobile terminal to the gateway
f^c_i	The computational resources allocated by the cloud for the execution of task z_i	$t^{down}_{i,tran}$	The transmission latency for downloading task z_i's data from the gateway to the mobile terminal
n	The number of tasks	$t^{c,up}_{i,tran}$	The transmission latency for uploading task z_i's data from the gateway to the cloud
m	The number of wireless networks	$t^{c,down}_{i,tran}$	The transmission latency for downloading task z_i's data from the cloud to the gateway
d_i	The input size of task z_i	$t^{up}_{CMT}(d_i)$, $t^{down}_{CMT}(\delta_i)$	The transmission latency for uploading d_i and downloading δ_i using CMT
c_i	The workload of task z_i	e^e_i, e^c_i	The energy consumption when task z_i is offloaded to the edge and cloud for execution
μ_i	The privacy level of task z_i	P_0	The idle power of the mobile terminal
δ_i	The output size of task z_i	H_i	The privacy risk of task z_i
k^l	The power coefficient of the mobile terminal	o_i	The deadline of task z_i
t^l_i	The computation latency when executing task z_i on the mobile terminal	$U_{i,sen}$, $U_{i,toler}$	The utilities of latency-sensitive and latency-tolerant task z_i
p^l_i	The power of mobile terminal when executing task z_i locally	η_1, η_2, η_3	The weights for three optimization objectives terminal
e^l_i	The energy consumption when executing task z_i on the mobile terminal	Δ_i	The cache size of task z_i
$t^e_{i,com}$	The computation latency when executing task z_i on the edge	R_e	The cache capacity of the edge
$t^c_{i,com}$	The computation latency when executing task z_i on the cloud	H	The privacy risk threshold
$\widetilde{W^{up}_{i,w}}$, $\widetilde{W^{down}_{i,w}}$	The uplink and downlink bandwidths allocated to task z_i by the wth wireless network	F^e	The computational resource capacity of the edge
$\widetilde{\Omega_w}$	The stochastic coefficient for the wth wireless network	a_w, b_w	The shape parameters of the Beta distribution
N_0	The Gaussian white noise density	s_0, s_n	The reference distance and distance between the mobile user and gateway
g^{up}_w, g^{down}_w	The uplink and downlink fading coefficients of the wth wireless network	φ_w	The path loss exponent of the wth wireless network
$P^{up}_{i,w}$, $P^{down}_{i,w}$	The uplink and downlink transmit power of the wth wireless network	$\widetilde{W^{up}_w}$, $\widetilde{W^{down}_w}$	The uplink and downlink bandwidths of the wth wireless network
$\mathbf{B}$, $\mathbf{A}$, $\mathbf{\Lambda}$	The task offloading strategy, task caching strategy, and CMT task data allocation strategy	W^{up}_w, W^{down}_w	The nominal uplink and downlink bandwidths of the wth wireless network

additional data transfer. Therefore, when making offloading strategies, it is imperative to evaluate the consumption associated with task execution in different locations. We define a binary matrix $\mathbf{B} = \left[\beta^l_i, \beta^e_i, \beta^c_i\right]_{n\times 3}$ to represent the task offloading strategy. Elements β^l_i, β^e_i, and β^c_i respectively indicate whether task z_i is executed locally, offloaded to the edge for execution, or offloaded to the cloud for execution. For example, $\beta^l_i = 1$ indicates that task z_i is executed locally, and $\beta^l_i = 0$ indicates that task z_i is not executed locally.

If task z_i is executed locally, its computation latency is expressed as

$$t^l_i = \frac{c_i}{f^l}, \tag{1}$$

in which f^l represents the computational resources of the mobile terminal. If task z_i is executed locally, the power of mobile terminal is denoted as

$$p^l_i = k^l (f^l)^3, \tag{2}$$

in which k^l is a constant that depends on the characteristics of CPU architecture in the mobile terminal. Eq. (2) adopts the widely used power consumption model, which has been validated by [44] and [45]. The energy consumption when task z_i is executed locally can be expressed as

$$e^l_i = p^l_i t^l_i = k^l (f^l)^2 c_i. \tag{3}$$

If task z_i is offloaded to the edge for execution, its computation latency is expressed as

$$t^e_{i,com} = \frac{c_i}{f^e_i}, \tag{4}$$

in which f^e_i represents the computational resources allocated by the edge for task z_i's execution. Despite the cloud theoretically possesses infinite resources, the concrete execution of task z_i still requires the allocation of quantifiable computational resources. Similarly, if task z_i is offloaded to the cloud for execution, its computation latency is expressed as

$$t^c_{i,com} = \frac{c_i}{f^c_i}, \tag{5}$$

in which f^c_i represents the computational resources allocated by the cloud for task z_i's execution.

3.1.4. Communication model

Real-life wireless networks exhibit inherent uncertainty and continuous fluctuations attributed to factors such as multi-path propagation, Doppler effect, shadowing effect, and interference. In this paper, we utilize stochastic parameters to characterize the uncertainty associated with wireless network bandwidth. We employ CMT to transmit data and assume the existence of m wireless networks. The uplink bandwidth $\widetilde{W^{up}_w}$ and downlink bandwidth $\widetilde{W^{down}_w}$ of the w-th ($1 \le w \le m$) wireless network are both stochastic numbers.

We define a matrix $\mathbf{\Lambda} = [\lambda^{up}_{i,w}, \lambda^{down}_{i,w}]_{2n\times m}$ ($\lambda_{i,w} \in [0,1]$) to represent the CMT task data allocation strategy. The element $\lambda^{up}_{i,w}$ ($\lambda^{down}_{i,w}$) represents the fraction of task z_i's data allocated for uploading (downloading) on the wth wireless network. According to Shannon's formula, the upload and download rates for transferring task z_i's data on the wth wireless network are respectively denoted as

$$r^{up}_{i,w} = \widetilde{W^{up}_{i,w}} \log_2 \left[1 + \frac{P^{up}_{i,w} g^{up}_w (s_0/s_n)^{\varphi_w}}{N_0 \widetilde{W^{up}_{i,w}}} \right] \tag{6}$$

and

$$r_{i,w}^{down} = \widetilde{W_{i,w}^{down}} \log_2 \left[1 + \frac{P_{i,w}^{down} g_w^{down} (s_0/s_n)^{\varphi_w}}{N_0 \widetilde{W_{i,w}^{down}}} \right]. \tag{7}$$

The uplink bandwidth allocated for uploading task z_i's data from the wth wireless network is represented by $\widetilde{W_{i,w}^{up}}$, while the downlink bandwidth is represented by $\widetilde{W_{i,w}^{down}}$. Both $\widetilde{W_{i,w}^{up}}$ and $\widetilde{W_{i,w}^{down}}$ are stochastic variables. $P_{i,w}^{up}$ ($P_{i,w}^{down}$) represents the transmit power of the wth wireless network for uploading (downloading) task z_i's data. g_w^{up} (g_w^{down}) represents the uplink (downlink) fading coefficient of the wth wireless network. φ_w represents the path loss exponent of the wth wireless network. s_0 represents the reference distance, s_n represents the distance between the mobile user and the gateway, and N_0 represents the Gaussian white noise density.

We define a binary vector $\boldsymbol{\Lambda} = [\alpha_i]_n$ to represent the task caching strategy. The element $\alpha_i = 1$ indicates that task z_i is cached at the edge, while $\alpha_i = 0$ indicates that task z_i is not cached at the edge. The data upload latency of task z_i is denoted as

$$t_{i,tran}^{up} = (1 - \alpha_i) t_{CMT}^{up}(d_i), \tag{8}$$

in which $t_{CMT}^{up}(d_i)$ represents the upload latency of d_i using CMT and is equal to the maximum latency among m wireless networks. $t_{CMT}^{up}(d_i)$ is expressed as

$$t_{CMT}^{up}(d_i) = \max \left\{ \frac{d_i \lambda_{i,1}^{up}}{r_{i,1}^{up}}, \frac{d_i \lambda_{i,2}^{up}}{r_{i,2}^{up}}, \ldots, \frac{d_i \lambda_{i,m}^{up}}{r_{i,m}^{up}} \right\}. \tag{9}$$

Despite the fact that edge possesses greater computational capabilities compared to the mobile terminal, its computational capacity is also limited. When the task requires significant computational power, MEC can opt to offload it to the cloud. The data upload latency of task z_i from the gateway to the cloud is expressed as

$$t_{i,tran}^{c,up} = \frac{d_i}{r^{c,up}}, \tag{10}$$

in which $r^{c,up}$ denotes the wired uplink transmission rate from the gateway to cloud. Similarly, the transmission latency for task z_i to be downloaded from the cloud to gateway is expressed as

$$t_{i,tran}^{c,down} = \frac{\delta_i}{r^{c,down}}, \tag{11}$$

in which $r^{c,down}$ denotes the wired downlink transmission rate from the cloud to gateway. The transmission latency for task z_i to be downloaded from the gateway to mobile terminal is expressed as

$$t_{i,tran}^{down} = t_{CMT}^{down}(\delta_i), \tag{12}$$

in which $t_{CMT}^{down}(\delta_i)$ represents the download latency of δ_i using CMT and is equal to the maximum latency among m wireless networks. $t_{CMT}^{down}(\delta_i)$ is expressed as

$$t_{CMT}^{down}(\delta_i) = \max \left\{ \frac{\delta_i \lambda_{i,1}^{down}}{r_{i,1}^{down}}, \frac{\delta_i \lambda_{i,2}^{down}}{r_{i,2}^{down}}, \ldots, \frac{\delta_i \lambda_{i,m}^{down}}{r_{i,m}^{down}} \right\}. \tag{13}$$

When task z_i is offloaded and executed at the edge or cloud, the mobile terminal is relieved from consuming energy for its execution. However, it may still incur energy consumption for uploading data, necessitating the calculation of its energy expenditure for data transmission. The energy consumption when task z_i is offloaded to the edge for execution is expressed as

$$e_i^e = (1 - \alpha_i) \sum_{w=1}^{m} P_{i,w}^{up} \frac{d_i \lambda_{i,w}^{up}}{r_{i,w}^{up}} + P_0 t_{i,com}^e, \tag{14}$$

in which P_0 represents the idle power of the mobile terminal. The energy consumption when task z_i is offloaded to the cloud for execution is expressed as

$$e_i^c = (1 - \alpha_i) \sum_{w=1}^{m} P_{i,w}^{up} \frac{d_i \lambda_{i,w}^{up}}{r_{i,w}^{up}} + P_0 (t_{i,tran}^{c,up} + t_{i,com}^c). \tag{15}$$

3.1.5. Privacy risk model

As tasks can be offloaded to the edge or cloud for execution, the transmission process becomes susceptible to malicious attacks and eavesdropping. Different tasks involve unique data, processes, and objectives, leading to diverse privacy requirements. We establish a privacy risk model to assess the risk of the task offloading process. In the DAG application model, greater emphasis is placed on the initial tasks as they often involve sensitive information. Therefore, in this paper, we define the privacy level of task z_i as

$$\mu_i = 1 - \log(1 + i). \tag{16}$$

MEC task offloading is considered secure if the cumulative privacy level falls below the predetermined threshold; otherwise, it is deemed insecure. We define the privacy risk as

$$H_i = \begin{cases} 0, \sum_{i=1}^{n} (\beta_i^e \vee \beta_i^c) \mu_i < \theta \\ \frac{1}{1 + e^{2\theta - \sum_{i=1}^{n} (\beta_i^e \vee \beta_i^c) \mu_i(i))}}, \sum_{i=1}^{n} (\beta_i^e \vee \beta_i^c) \mu_i \geq \theta \end{cases}, \tag{17}$$

in which $\vee$ represents the Boolean OR operator. When $\beta_i^e \vee \beta_i^c = 1$, it signifies the offloading of task z_i to the edge or cloud, whereas $\beta_i^e \vee \beta_i^c = 0$ indicates the local execution of task z_i.

3.1.6. Task utility model

The principal goals of task offloading commonly center on diminishing latency or energy consumption. Nonetheless, the satisfaction of tasks following offloading is equally indispensable and warrants careful consideration. We employ task utility as a representation of the satisfaction. Tasks are classified into latency-sensitive and latency-tolerant categories, given their distinct preferences for satisfaction. Latency-sensitive tasks must be completed within specific time frames. With an increase in the execution latency, user experience is adversely affected, resulting in a decline in satisfaction. The S-shaped function is well-suited for effectively characterizing the utility [49]. The utility function of the latency-sensitive task z_i is expressed as

$$U_{i,sen} = \begin{cases} \frac{1}{1 + e^{-\xi_1 (o_i - t_i)}}, t_i \leq o_i \\ 1 - e^{\xi_4 (t_i - o_i)}, t_i > o_i \end{cases}. \tag{18}$$

Latency-tolerant tasks can be delayed within a certain time range, and we employ a logarithmic function to describe the utility of these tasks. The utility function of latency-tolerant task z_i is expressed as

$$U_{i,toler} = \begin{cases} 1 - \xi_2 \log (1 + \xi_3 o_i), t_i \leq o_i \\ 1 - e^{\xi_4 (t_i - o_i)}, t_i > o_i \end{cases}. \tag{19}$$

ξ_1, ξ_2, ξ_3 and ξ_4 are parameters that can be adjusted according to the specific characteristics of the task, providing a more accurate description of the satisfaction. o_i represents the deadline for task z_i. t_i represents the execution latency of task z_i, defined as the time required for the completion of task z_i, encompassing both its inherent execution latency and the transmission latency. t_i is expressed as

$$\begin{aligned} t_i \; = & \beta_i^l t_i^l + (\beta_i^e \vee \beta_i^c) t_{i,tran}^{up} + \beta_i^e t_{i,com}^e + \beta_i^c (t_{i,tran}^{c,up} + t_{i,com}^c) + \sum_{j=1}^{n} \tau_{j,i} \\ & \times \begin{pmatrix} (\beta_j^l \wedge \beta_i^e) t_{CMT}^{up}(\delta_j) + (\beta_j^l \wedge \beta_i^c)(t_{CMT}^{up}(\delta_j) + \frac{\delta_j}{r^{c,up}}) + (\beta_j^e \wedge \beta_i^c) \frac{\delta_j}{r^{c,up}} \\ + (\beta_j^e \wedge \beta_i^l) t_{j,tran}^{down} + (\beta_j^c \wedge \beta_i^l)(t_{j,tran}^{down} + t_{j,tran}^{c,down}) + (\beta_j^c \wedge \beta_i^e) t_{j,tran}^{c,down} \end{pmatrix}, \end{aligned} \tag{20}$$

in which $\wedge$ represents the boolean AND operator. The result of $\wedge$ takes the value of 1 if the two tasks are offloaded to distinct locations. For example, $\beta_j^l \wedge \beta_i^e = 1$ indicates that task z_j is executed locally, while task z_i is offloaded to the edge for execution.

3.2. Problem formulation and proof

3.2.1. Problem formulation

This paper aims to achieve three objectives: minimizing latency, minimizing energy consumption, and maximizing task utility. To consolidate these three optimization objectives, we employ a linear weighting method. However, since these objectives possess different units, we must normalize them for meaningful comparison and analysis.

For the application model adopted in this paper, the total latency is expressed as

$$t = \sum_{i=1}^{n} t_i. \tag{21}$$

Meanwhile, for general DAG application models, the latency is determined by the critical path, and the total latency is expressed as

$$t = \max_{\varsigma \in p} \left\{ \sum_{i \in \varsigma} t_i \right\}. \tag{22}$$

p denote the set of all paths from the start task to the end task, where $\varsigma \in p$ represents one such path. When using a general DAG application model, it suffices to replace formula (21) with formula (22). The normalized latency is expressed as

$$T = \frac{t}{t_{local}}, \tag{23}$$

in which $t_{local} = \sum_{i=1}^{n} t_i^l$ denotes the latency incurred when all tasks are executed on the mobile terminal. The total energy consumption resulting from all tasks is expressed as

$$\begin{aligned} e = & \sum_{i=1}^{n} (\beta_i^l e_i^l + \beta_i^e e_i^e + \beta_i^c e_i^c) + \sum_{j=1}^{n} \sum_{i=1}^{n} \tau_{j,i} \\ & \times \left(\begin{array}{l} (\beta_j^l \wedge \beta_i^e) \sum_{w=1}^{m} P_{j,w}^{up} \frac{\delta_j \lambda_{j,w}}{r_{j,w}^{up}} + (\beta_j^l \wedge \beta_i^c)(\sum_{w=1}^{m} P_{j,w}^{up} \frac{\delta_j \lambda_{j,w}}{r_{j,w}^{up}} \\ + P_0 \frac{\delta_j}{r^{c,up}}) + (\beta_j^e \wedge \beta_i^c) P_0 \frac{\delta_j}{r^{c,up}} + (\beta_j^e \wedge \beta_i^l) P_{j,w}^{down} t_{j,tran}^{down} \\ + (\beta_j^c \wedge \beta_i^l)(P_{j,w}^{down} t_{j,tran}^{down} + P_0 t_{j,tran}^{c,down}) + (\beta_j^c \wedge \beta_i^e) P_0 t_{j,tran}^{c,down} \end{array} \right). \end{aligned} \tag{24}$$

The normalized energy consumption is expressed as

$$E = \frac{e}{e_{local}}, \tag{25}$$

in which $e_{local} = \sum_{i=1}^{n} e_i^l$ denotes the energy consumption incurred when all tasks are executed on the mobile terminal. The total task utility of all tasks is expressed as

$$q = \sum_{z_i \in N_1} U_{i,sen} + \sum_{z_j \in N_2} U_{j,toler}, \tag{26}$$

in which N_1 and N_2 represent the set of latency-sensitive and latency-tolerant tasks, respectively. The normalized task utility is expressed as

$$Q = \frac{q - Q_{\min}}{Q_{\max} - Q_{\min}}. \tag{27}$$

in which $Q_{\min}$ and $Q_{\max}$ denote the minimum and maximum value of the total task utility, respectively.

The objective function based on linear weighting is expressed as

$$\begin{aligned} \widetilde{F} &= \eta_1 T + \eta_2 E - \eta_3 Q \\ &= \eta_1 \frac{t}{t_{local}} + \eta_2 \frac{e}{e_{local}} - \eta_3 \frac{q - Q_{\min}}{Q_{\max} - Q_{\min}}, \end{aligned} \tag{28}$$

in which η_1, η_2, and η_3 ($\eta_1, \eta_2, \eta_3 \in [0, 1]$, $\eta_1 + \eta_2 + \eta_3 = 1$) represent the weights of latency, energy consumption, and task utility, respectively. These parameters enable the adjustment of the relative importance of the three objectives, allowing for varying proportions based on specific requirements. The objective function $\widetilde{F}$ exhibits stochasticity due to the presence of stochastic parameters within T, E, and Q.

The task offloading problem in this paper is formulated as

$$\min_{\mathbf{B},\mathbf{A},\mathbf{\Lambda}} \widetilde{F} \tag{29}$$

s.t.

$$\forall w \in [1, m], \sum_{i=1}^{n} (\beta_i^e \vee \beta_i^c)(1 - \alpha_i) \widetilde{W_{i,w}^{up}} + \sum_{i=1}^{n} \sum_{j=1}^{n} \tau_{i,j} (\beta_i^l \wedge (\beta_j^e \vee \beta_j^c)) \widetilde{W_{j,w}^{up}} \leq \widetilde{W_w^{up}} \tag{30}$$

$$\forall w \in [1, m], \sum_{i=1}^{n} \sum_{j=1}^{n} \tau_{i,j} (\beta_j^l \wedge (\beta_i^e \vee \beta_i^c)) \widetilde{W_{j,w}^{down}} \leq \widetilde{W_w^{down}} \tag{31}$$

$$\sum_{i=1}^{n} H_i \leq H \tag{32}$$

$$\sum_{i=1}^{n} \alpha_i \Delta_i \leq R^e \tag{33}$$

$$\sum_{i=1}^{n} \beta_i{}^e f_i^e \leq F^e \tag{34}$$

$$\forall i \in [1, n], \sum_{w=1}^{m} \lambda_{i,w}^{up} = \sum_{w=1}^{m} \lambda_{i,w}^{down} = 1 \tag{35}$$

$$\forall i \in [1, n], \beta_i{}^l + \beta_i{}^e + \beta_i{}^c = 1 \tag{36}$$

$$\forall i \in [1, n], \beta_i^l, \beta_i^e, \beta_i^c, \alpha_i \in \{0, 1\} \tag{37}$$

$$\forall i \in [1, n], \forall w \in [1, m], 0 \leq \lambda_{i,w}^{up}, \lambda_{i,w}^{down} \leq 1 \tag{38}$$

Formula (29) aims to minimize the objective function. Constraint (30) guarantees that the uplink bandwidth allocated from the wth wireless network does not exceed its capacity $\widetilde{W_w^{up}}$. Similarly, constraint (31) guarantees that the downlink bandwidth allocated from the wth wireless network does not exceed its capacity $\widetilde{W_w^{down}}$. Constraint (32) ensures that the privacy risk remains below the specified risk threshold H during the task offloading process. Constraint (33) ensures that the cache space of the edge is not exceeded by the cached tasks, where Δ_i represents the cache size of task z_i and R^e represents the cache capacity of the edge. Despite the superior resource capabilities of edge servers compared to mobile terminals, inherent limitations exist. Constraint (34) ensures that the allocation of computational resources by the edge does not exceed its capacity F^e. Constraint (35) guarantees the complete data transmission for task z_i. Constraint (36) guarantees that each task can only be offloaded to one location for execution. Constraint (37) enforces that β_i^l, β_i^e, β_i^c, and α_i can only be binary values of 0 or 1. Constraint (38) ensures the continuity of $\lambda_{i,w}^{up}$ and $\lambda_{i,w}^{down}$ within the range of 0 to 1.

3.2.2. NP-hardness proof

We prove the NP-hardness of the established task offloading model by constructing a reduction to the classical NP-hard bin packing problem. Specifically, we show that the reduced task offloading model can be equivalently formulated as a bin packing problem. The complete task offloading optimization model incorporates uncertainties and encompasses multiple decision-making aspects beyond the reduced model. This reduced model corresponds to an instance snapshot within a family of

infinitely many such reduced models subsumed by the complete model. The greater complexity of the complete model implies that our problem is at least as hard as bin packing. The detailed proof is as follows.

Theorem 1. The established task offloading optimization model is NP-hard.

Proof: The bin packing model is a well-known NP-hard optimization problem, where a set of items is allocated into several bins, subject to the constraint of not exceeding the bin capacities, in order to optimize the objective function [50]. The bin packing model can be expressed as

$$\min \sum_{i=1}^{I}\sum_{j=1}^{J} f(y_{i,j}) \tag{39}$$

s.t.

$$\forall j \in [1,J], \sum_{i=1}^{I} \pi_i y_{i,j} \le \Pi_j \tag{40}$$

$$\forall i \in [1,I], \sum_{j=1}^{J} y_{i,j} = 1 \tag{41}$$

$$\forall i \in [1,I], \forall j \in [1,J], y_{i,j} \in \{0,1\} \tag{42}$$

Formula (39) represents the objective function. $y_{i,j}$ is a binary variable that signifies whether the ith item is packed into the jth bin. Constraint (40) guarantees that the occupied size of packed items in the jth bin does not exceed its capacity, in which π_i represents the size of the ith item, and Π_j represents the capacity of the jth bin. Constraint (41) guarantees that each item can only be packed into a single bin. Constraint (42) ensures that the variable $y_{i,j}$ can only take binary values of 0 or 1.

We consider a reduced instance of our task offloading optimization model, wherein the uncertainty is disabled, as well as the objectives of minimizing energy consumption and maximizing task utility, and the joint of CMT, task caching, and privacy protection. In the reduced model, there are n tasks and three potential locations (mobile terminal, edge, and cloud) to execute the task. The objective is to minimize the latency by either locally executing the tasks or offloading them to the edge or cloud. We consider the n tasks as n items and treat the three potential locations as three distinct bins. The mobile terminal corresponds to the first bin, the edge corresponds to the second bin, and the cloud corresponds to the third bin. The reduced model can be formulated as a bin packing problem, where the n items (tasks) are allocated (offloaded) to three distinct bins (mobile terminal, edge, and cloud).

$T_1 = \sum_{i=1}^{n} t_i^l$, $T_2 = \sum_{i=1}^{n} t_{i,com}^e$, and $T_3 = \sum_{i=1}^{n} t_{i,com}^c$ denote the latency associated with executing n tasks on the mobile terminal, edge, and cloud, respectively. The offloading strategy is equivalent to the binary variable $\beta_{i,j}$, that is $\beta_{i,1} = \beta_i^l$, $\beta_{i,2} = \beta_i^e$, and $\beta_{i,3} = \beta_i^c$. Let $t_{i,1}$, $t_{i,2}$, and $t_{i,3}$ denote the latency associated with executing the ith task on the mobile terminal, edge, and cloud, respectively. $t_{i,1}$, $t_{i,2}$, and $t_{i,3}$ are expressed as

$$t_{i,1} = t_i^l + \sum_{k=1}^{n} \tau_{k,i} \Big((\beta_{k,3} \wedge \beta_{i,1})(t_{k,tran}^{down} + t_{k,tran}^{c,down}) + (\beta_{k,2} \wedge \beta_{i,1}) t_{k,tran}^{down} \Big), \tag{43}$$

$$t_{i,2} = t_{i,com}^e + t_{i,tran}^{up} + \sum_{k=1}^{n} \tau_{k,i} \Big((\beta_{k,1} \wedge \beta_{i,2}) \tfrac{\delta_k}{r_{i,1}^{up}} + (\beta_{k,3} \wedge \beta_{i,2}) t_{k,tran}^{c,down} \Big), \tag{44}$$

and

$$\begin{aligned} t_{i,3} = {} & t_{i,com}^c + t_{i,tran}^{up} + t_{i,tran}^{c,up} \\ & + \sum_{k=1}^{n} \tau_{k,i} \Big((\beta_{k,1} \wedge \beta_{i,3})(\tfrac{\delta_k}{r_{i,1}^{up}} + \tfrac{\delta_k}{r^{c,up}}) + (\beta_{k,2} \wedge \beta_{i,3}) \tfrac{\delta_k}{r^{c,up}} \Big). \end{aligned} \tag{45}$$

Subsequently, the reduced task offloading optimization model can be formulated as

$$\min \sum_{i=1}^{n}\sum_{j=1}^{3} t_{i,j}\beta_{i,j} \tag{46}$$

s.t.

$$\forall j \in [1,3], \sum_{i=1}^{n} t_{i,j}\beta_{i,j} \le T_j \tag{47}$$

$$\forall i \in [1,n], \sum_{j=1}^{3} \beta_{i,j} = 1 \tag{48}$$

$$\forall i \in [1,n], \forall j \in [1,3], \beta_{i,j} \in \{0,1\} \tag{49}$$

Formula (46) represents the objective function aimed at minimizing the latency. Constraint (47) guarantees that the latency incurred by tasks executed on the mobile terminal, edge, or cloud does not exceed the overall latency of n tasks executed across these locations. Constraint (48) ensures that each task can only be executed on a single location. Constraint (49) guarantees that the variables are restricted to binary values.

Formulas (46)–(49) establish the mathematical equivalence between the reduced task offloading optimization model and the bin packing model. The complete task offloading optimization model integrates uncertainties and includes the decision-making aspects of CMT task data allocation, task caching, and privacy protection constraints, in addition to the reduced task offloading optimization model. As a result, the complete model exhibits substantially higher complexity compared to the reduced model, which can be equivalently formulated as the NP-hard bin packing model represented by formulas (46)-(49). Consequently, the established task offloading optimization model is demonstrated to be NP-hard.

4. Description of the proposed offloading algorithm

4.1. Model decomposition and algorithm overview

4.1.1. Model decomposition

It is difficult to solve the established optimization model due to the existence of dependencies among unknown strategies. For instance, the knowledge of the offloading strategy $\mathbf{B}$ is necessary to determine the execution locations of each task, which subsequently enables the formulation of the CMT data allocation strategy $\mathbf{\Lambda}$. Simultaneously, it can be observed that the offloading strategy $\mathbf{B}$ and caching strategy $\mathbf{A}$ are discrete, whereas the allocation strategy $\mathbf{\Lambda}$ is continuous. This implies that handling the offloading location, caching location, and CMT data allocation ratio as separate entities not only simplifies the joint task offloading problem but also enhances the solving capability and efficiency of the offloading algorithm. Therefore, we decompose the model into two sub-problems named SP1 and SP2. The sub-problem SP1 is formulated as the objective function (29) with a given offloading strategy and caching strategy, subject to constraints (30), (31), (35), and (38). The sub-problem SP2 is formulated as the objective function (29) subject to constraints (30), (31), (32), (33), (34), (36), and (37).

We employ chance constraints to handle the uncertainties in the objective function (29), constraints (30) and (31). The chance constraint-based objective function can be expressed as

$$\min_{\mathbf{B},\mathbf{A},\mathbf{\Lambda}} F \quad s.t. \quad Pr\{\widetilde{F} \le F\} \ge l_F, \tag{50}$$

which ensures that the objective function reaches F at the confidence level l_F and converts the optimization objective to the minimization of F. $Pr\{\bullet\}$ represents the probability function that quantifies the likelihood of satisfying the input condition. The chance constraint-based constraints (30) and (31) can be expressed as

$$\begin{aligned} \forall w \in [1,m], Pr\Big\{ & \sum_{i=1}^{n} (\beta_i^e \vee \beta_i^c)(1-\alpha_i)\widetilde{W_{i,w}^{up}} \\ & + \sum_{i=1}^{n}\sum_{j=1}^{n} \tau_{i,j}(\beta_i^l \wedge (\beta_j^e \vee \beta_j^c))\widetilde{W_{j,w}^{up}} - \widetilde{W_w^{up}} \le 0 \Big\} \ge l_w^{up} \end{aligned} \tag{51}$$

and

$$\forall w \in [1,m], Pr\left\{ \sum_{i=1}^{n}\sum_{j=1}^{n} \tau_{i,j}(\beta_j^l \wedge (\beta_i^e \vee \beta_i^c))\widetilde{W_{j,w}^{down}} - \widetilde{W_w^{down}} \le 0 \right\} \ge l_w^{down}. \tag{52}$$

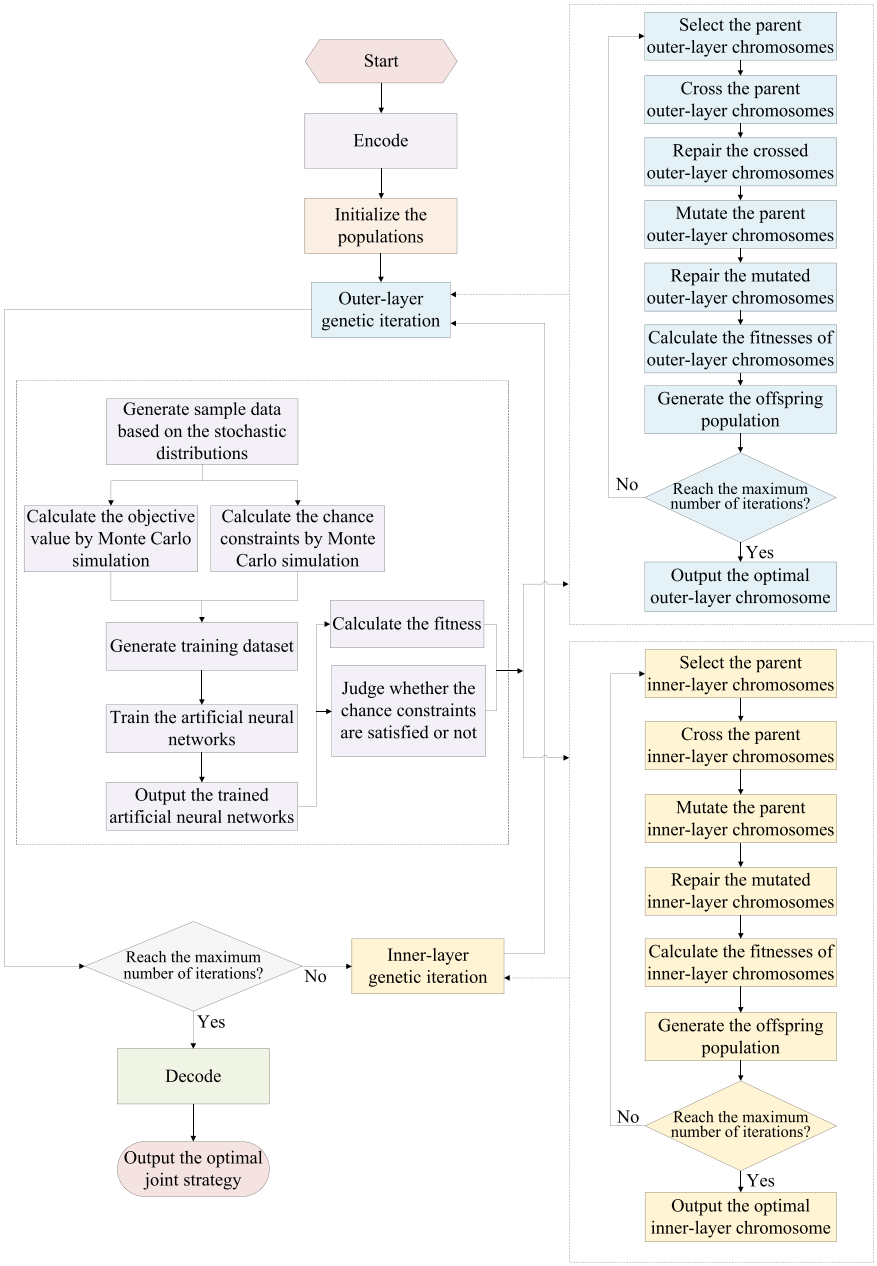


Fig. 2. The flow chart of the proposed hybrid two-layer offloading algorithm.

The chance constraint guarantees the fulfillment of these two types of constraints with confidence levels of l_w^{up} and l_w^{down}, respectively. By employing chance constraints to address the uncertainties in the task offloading model, we convert the resolution of the stochastic optimization model into solving a deterministic problem at specified confidence levels.

4.1.2. Algorithm overview

Building upon the two-layer decomposition of the model, we propose a hybrid two-layer offloading algorithm based on an improved genetic algorithm, Monte Carlo simulation, and artificial neural networks. The flow chart of the proposed hybrid two-layer offloading algorithm is illustrated in Fig. 2. The proposed offloading algorithm consists of outer and inner layers. The outer layer is specifically devised to address sub-problem SP2, which entails making decisions regarding offloading and caching. The inner layer is dedicated to solving sub-problem SP1, which involves formulating the CMT data allocation strategy based on the given offloading and caching strategies from the outer-layer algorithm. The outer and inner layers are depicted in Algorithms 1 and 2, respectively. The inner and outer layers of the offloading algorithm utilize the genetic algorithm framework as their skeleton, leveraging its search capabilities to explore optimal strategies. Improvements are made to the initialization and genetic operations within both layers, further enhancing the search efficiency of the offloading algorithm. To effectively address chance constraints, we employ the integration of Monte Carlo simulation with the artificial neural network. The outer-layer algorithm calls the inner-layer algorithm and provides its chromosome as an input. Given an outer-layer chromosome, the inner-layer algorithm performs genetic iterations to search for the corresponding optimal inner-layer chromosome. In the genetic iterations for searching the optimal outer-layer chromosome, fitness is calculated by the outer-layer algorithm based on the optimal inner-layer chromosome returned by the inner-layer algorithm. The optimal objective value is achieved only when both

Algorithm 1 The outer layer of the proposed offloading algorithm.

Input: population po, crossover probability C_p, mutation probability M_p
Output: optimal outer-layer chromosome

repeat
  call Algorithm 3
until po is initialized
call Algorithm 2 to obtain the optimal inner-layer chromosome for each outer-layer chromosome
calculate the fitness for each outer-layer chromosome by Eq. (53) with the artificial neural network trained by Algorithm 7 taking its optimal inner-layer chromosome as input
select two outer-layer chromosomes by the roulette wheel method
$r_1 \leftarrow rand(0,1)$, $r_2 \leftarrow rand(0,1)$
if $r_1 \leq C_p$ **then**
  cross in conjunction with the repair mechanism Algorithm 4
end if
if $r_2 \leq M_p$ **then**
  mutate in conjunction with the repair mechanism Algorithm 5
end if
call Algorithm 2 to obtain the optimal inner-layer chromosomes for these two new outer-layer chromosomes
calculate the fitness f_1 and f_2 for these two new outer-layer chromosomes by Eq. (53) with the artificial neural network trained by Algorithm 7 taking their optimal inner-layer chromosomes as input
$ri_1, ri_2 \leftarrow -1$, $mf_1, mf_2 \leftarrow 0$
for i in po **do**
  if $f_i - f_1 > mf_1$ **then**
    $mf_1 \leftarrow f_i - f_1$, $ri_1 \leftarrow i$
  end if
end for
for i in po **do**
  if $f_i - f_2 > mf_2$ **and** $i \neq ri_1$ **then**
    $mf_2 \leftarrow f_i - f_2$, $ri_2 \leftarrow i$
  end if
end for
if $ri_1 \geq 0$ **then**
  replace the outer-layer chromosome at position ri_1 in po
end if
if $ri_2 \geq 0$ **then**
  replace the outer-layer chromosome at position ri_2 in po
end if
repeat
  4–32
until the maximum number of iterations is reached

Algorithm 2 The inner layer of the proposed offloading algorithm.

Input: population po, crossover probability C_p, mutation probability M_p, the outer-layer chromosome
Output: optimal inner-layer chromosome

select two inner-layer chromosomes by the roulette wheel method
$r_1 \leftarrow rand(0,1)$, $r_2 \leftarrow rand(0,1)$
if $r_1 \leq C_p$ **then**
  Cross these two inner-layer chromosomes
end if
if $r_2 \leq M_p$ **then**
  Mutate in conjunction with the repair mechanism of Algorithm 6
end if
calculate the fitness f_1 and f_2 for these two new inner-layer chromosomes by Eq. (53) with the artificial neural network trained by Algorithm 7 taking the outer-layer chromosome as input
$ri_1, ri_2 \leftarrow -1$, $mf_1, mf_2 \leftarrow 0$
for i in po **do**
  if $f_i - f_1 > mf_1$ **then**
    $mf_1 \leftarrow f_i - f_1$, $ri_1 \leftarrow i$
  end if
end for
for i in po **do**
  if $f_i - f_2 > mf_2$ **and** $i \neq ri_1$ **then**
    $mf_2 \leftarrow f_i - f_2$, $ri_2 \leftarrow i$
  end if
end for
if $ri_1 \geq 0$ **then**
  replace the inner-layer chromosome at position ri_1 in po
end if
if $ri_2 \geq 0$ **then**
  replace the inner-layer chromosome at position ri_2 in po
end if
repeat
  1–26
until the maximum number of iterations is reached

the outer-layer and inner-layer chromosomes simultaneously reach their optimal states.

It is worth noting that there exists a preparation phase prior to the actual deployment of the offloading algorithm, during which the artificial neural networks are trained using data generated by Monte Carlo simulations. Algorithmic complexity is intended to characterize the performance of an algorithm over long-term operation. The computational overhead incurred in the preparation phase is a one-time cost. Once the artificial neural networks are trained, they can theoretically be reused indefinitely, rendering the amortized overhead negligible. Accordingly, our complexity analysis focuses on the decision-making phase (two-layer offloading algorithm), which governs the runtime behavior of the offloading algorithm in practical applications. The genetic algorithm employed in the outer-layer algorithm plays a crucial role in exploring the solution space for offloading and caching. As a result, the running time of the outer-layer algorithm is primarily determined by the genetic algorithm. The number of cycles performed by the outer-layer algorithm can be represented as $\Psi_1\Phi_1$, where Ψ_1 represents the maximum number of iterations, and Φ_1 represents the population size. The artificial neural networks are employed to obtain values of the fitness function and chance constraints, and their computational time is denoted as Θ. The computational time of the outer-layer algorithm can be expressed as $\Psi_1\Phi_1\Theta$. Similarly, the computational time of the inner-layer algorithm can be expressed as $\Psi_2\Phi_2\Theta$. The cumulative computational time of the proposed two-layer offloading algorithm is expressed as $\Phi_1\Phi_2\Theta^2\Psi_1\Psi_2$. Φ_1 and Φ_2 are two given constant coefficients, while Θ represents a numerically stable value that remains independent of the number of cycles. Therefore, the proposed two-layer offloading algorithm has time complexity $O(\Psi^2)$.

4.2. Encoding and fitness function

The strategy is encoded as the chromosome, and genetic operations are employed to iteratively modify these chromosomes, thereby searching for the optimal strategy. The inner-layer algorithm is responsible for solving the CMT data allocation strategy, where each chromosome within this layer represents a potential solution. Float-encoding is utilized in the inner-layer algorithm, and the chromosome is encoded as $2m$ levels shown in Fig. 3. The $(2w-1)$th and $2w$th levels respectively represent the upload and download data allocation strategies for the wth wireless network. The outer-layer algorithm is responsible for resolving both the offloading strategy and caching strategy. The chromosome within this layer is encoded with two levels shown in Fig. 4, wherein the first level signifies the offloading strategy and the second level signifies the caching strategy. The first level of the chromosome employs integer encoding, wherein the values “0”, “1”, and “2” respectively denote local

0.321	0.005	⋯	0.735
0.045	0.225	⋯	0.012
⋮	⋮	⋮	⋮
0.133	0.322	⋯	0.063

Fig. 3. The chromosome encoding in the inner-layer algorithm.

0	2	0	1	⋯	2
0	1	0	1	⋯	0

Fig. 4. The chromosome encoding in the outer-layer algorithm.

execution, offloading to the edge, and offloading to the cloud. The second level of the chromosome utilizes binary encoding, where the values "0" and "1" respectively indicate uncached and cached states.

The fitness function is used to evaluate the adaptability of each chromosome in the population. Genetic algorithms employ fitness functions to perform operations such as selection, crossover, and mutation based on the fitness values assigned to chromosomes, thereby progressively optimizing the chromosomes within the population. The fitness function is defined as

$$f = \frac{1}{F}. \tag{53}$$

4.3. Improved initialization

The initialization process establishes an initial population, serving as the starting point for exploring the search space. The quality of initialization significantly influences the algorithm's performance and convergence speed. Traditional initialization randomly generates a set of initial chromosomes. However, this initialization method generates a large number of infeasible chromosomes when facing the established task offloading model with complex solution spaces and constraints, resulting in low convergence efficiency of the offloading algorithm and an increased likelihood of becoming trapped in local optima. Therefore, we improve the initialization shown in Algorithm 3 to enhance the efficiency and mitigate risk of converging to local optima. The improved initialization guarantees the feasibility of all chromosomes, thereby preventing the generation of solutions that fail to meet the constraints.

Algorithm 3 The improved initialization.

Input: population po, edge caching threshold S_t, task set N, task cache size set $S = \{s_1, s_2, \ldots, s_n\}$
Output: outer-layer chromosome x^O, inner-layer chromosome x^I

$t \leftarrow 0$
for i in po **do**
  for $j = 1$ to n **do**
    $x_i^O[0][j] \leftarrow rand(0,1)$
    if ($x_i^O[0][j] \geq 0$ **and** $x_i^O[0][j] < \frac{1}{3}$) **or** (task must be executed locally) **then**
      $x_i^O[0][j] \leftarrow 0$
    else if $x_i^O[0][j] \geq \frac{1}{3}$ **and** $x_i^O[0][j] < \frac{2}{3}$ **then**
      $x_i^O[0][j] \leftarrow 1$
    else
      $x_i^O[0][j] \leftarrow 2$
    end if
  end for
end for
for i in po **do**
  while $N \neq \emptyset$ **do**
    select task z_j from N randomly
    if $x_i^O[0][j] = 1$ **or** $x_i^O[0][j] = 2$ **then**
      $x_i^O[1][j] \leftarrow 1$
    end if
    if $x_i^O[1][j] = 1$ **then**
      if $t + s_j > S_t$ **then**
        $x_i^O[1][j] \leftarrow 0$
      else
        $t \leftarrow t + s_j$
        delete task z_j from N
      end if
    end if
  end while
end for
for i in po **do**
  for $j = 1$ to n **do**
    if $x_i^O[0][j] = 1$ **or** $x_i^O[0][j] = 2$ **then**
      $sum_i^I[j] \leftarrow 0$
      for $w = 1$ to m **do**
        $x_i^I[w][j] \leftarrow rand(0, 1 - sum_i^I[j])$
        $sum_i^I[j] \leftarrow sum_i^I[j] + x_i^I[w][j]$
      end for
    else
      $x_i^I[][j] \leftarrow 0$
    end if
  end for
end for

4.4. Genetic operations based on repair mechanisms

4.4.1. Selection

The selection operation holds significant importance in genetic algorithms due to its crucial role in shaping the population and driving the evolutionary process. It selects chromosomes for crossover and mutation according to the chromosome fitness. In this paper, roulette wheel selection is employed, wherein the probability of selecting each chromosome is directly proportional to its fitness value. The roulette wheel selection allows chromosomes with higher fitness values to have a greater chance of being chosen as parents for the next generation.

4.4.2. Crossover and mutation

The fundamental concept underlying crossover entails the selection of two parent chromosomes, followed by the exchange of partial genes between them to generate offspring chromosomes. Mutation involves the random alteration of genes at specific loci within the chromosome. By introducing these random changes, mutation serves to enhance both chromosome diversity and the exploration of the solution space. However, in the presence of multiple constraints within the established model, relying exclusively on basic crossover and mutation operations to generate new chromosomes may result in solutions that violate these constraints, rendering them infeasible. Therefore, to address the issue of generating infeasible chromosomes through basic crossover and mutation operations, we propose some repair mechanisms. These mechanisms aim to rectify the infeasibility of newly generated chromosomes by modifying or adjusting their genetic makeup, ensuring compliance with the specified constraints.

The repair mechanisms proposed for the outer-layer algorithm are demonstrated in Algorithms 4 and 5. The repair mechanisms play a critical role in ensuring adherence to cache space constraints within the cache encoding of outer-layer chromosomes during the processes of crossover and mutation. To rectify any violations of this constraint, the repair mechanisms employ two well-defined approaches involving gene adjustments and strategic random replacements, respectively. By systematically modifying the genetic makeup of the affected chromosomes, the repair mechanisms effectively restore the integrity and conformity of the chromosomes within specified constraints, thereby preserving the

Algorithm 4 Repair mechanism for crossover in the outer-layer algorithm.

Input: population po, edge caching threshold S_t, task set N, task cache size set $S = \{s_1, s_2, \ldots, s_n\}$, two parent outer-layer chromosome levels $y_1^O[1]$ and $y_2^O[1]$
Output: two new outer-layer chromosome levels $x_1^O[1]$ and $x_2^O[1]$

$C \leftarrow randInt(0, n)$
$x_1^O[1], x_2^O[1] \leftarrow \emptyset$, $t_1, t_2 \leftarrow 0$
for $j = 1$ to C **do**
  $x_1^O[1][j] \leftarrow y_1^O[1][j]$
  if $x_1^O[1][j] = 1$ **then**
    $t_1 \leftarrow t_1 + s_j$
  end if
  $x_2^O[1][j] \leftarrow y_2^O[1][j]$
  if $x_2^O[1][j] = 1$ **then**
    $t_2 \leftarrow t_2 + s_j$
  end if
end for
for $j = C$ to n **do**
  $x_1^O[1][j] \leftarrow y_2^O[1][j]$
  if $x_1^O[1][j] = 1$ **then**
    if $t_1 + s_i > S_t$ **then**
      $x_1^O[1][j] \leftarrow 0$
    else
      $t_1 \leftarrow t_1 + s_j$
    end if
  end if
  $x_2^O[1][j] \leftarrow y_1^O[1][j]$
  if $x_2^O[1][j] = 1$ **then**
    if $t_2 + s_j > S_t$ **then**
      $x_2^O[1][j] \leftarrow 0$
    else
      $t_2 \leftarrow t_2 + s_j$
    end if
  end if
end for

validity and reliability of the overall algorithmic framework. The repair mechanism designed for the inner-layer algorithm is illustrated in Algorithm 6. This algorithm is specifically devised to address violations of wireless network capacity constraints within the inner-layer chromosomes during the mutation process. By employing gene pruning, the repair mechanism effectively restores the integrity of inner-layer chromosomes, ensuring adherence to the specified capacity constraints.

4.5. Chance constraint processing based on monte carlo simulation and artificial neural networks

Monte Carlo simulation is a widely utilized numerical calculation method rooted in random sampling and statistical principles, extensively applied to address problems involving uncertainty. To deal with chance constraints, this paper adopts the Monte Carlo simulation method, which generates random numbers to calculate the objective function and constraint satisfaction probability. The fundamental concept underlying Monte Carlo simulation is to convert problems involving uncertainty into deterministic sampling problems, which can then be approximated through a substantial amount of simple data. Monte Carlo simulation enables the investigation and analysis of complex systems by generating a multitude of random samples to represent the uncertain parameters involved. Monte Carlo simulation yields more accurate results only when a sufficient number of simulations are conducted. Nonetheless, the utilization of Monte Carlo simulation substantially amplifies the time complexity. Additionally, the requirement to re-run Monte Carlo simulation whenever there is a change in input parameters leads to a significant amount of redundant computation.

Algorithm 5 Repair mechanism for mutation in the outer-layer algorithm.

Input: population po, edge caching threshold S_t, task set N, task cache size set $S = \{s_1, s_2, \ldots, s_n\}$, parent outer-layer chromosome level $y_1^O[1]$
Output: new chromosome level $x_1^O[1]$

$M \leftarrow randInt(0, n)$
if $x_1^O[1][M] = 1$ **then**
  $x_1^O[1][M] \leftarrow 0$
else
  $x_1^O[1][M] \leftarrow 1$
end if
if $x_1^O[1][M] = 1$ **then**
  $t_1 \leftarrow t_1 + s_M$
end if
while $t_1 > S_t$ **do**
  select task z_j from N randomly
  if $x_1^O[1][j] = 1$ **then**
    $x_1^O[1][j] \leftarrow 0$
    $t_1 \leftarrow t_1 - s_j$
  end if
end while

Algorithm 6 Repair mechanism for mutation in the inner-layer algorithm.

Input: population po, task set N, parent inner-layer chromosome y_1^I
Output: new inner-layer chromosome x_1^I

$M \leftarrow randInt(0, n)$
while $y_i^O[0][M] \neq 0$ **do**
  $x_1^I \leftarrow \emptyset$
  $sum_1^I[j] \leftarrow 0$
  for $w = 1$ to m **do**
    $x_1^I[w][j] \leftarrow rand(0, 1 - sum_1^I[j])$
    $sum_1^I[j] \leftarrow sum_1^I[j] + x_1^I[w][j]$
  end for
  $M \leftarrow randInt(0, n)$
end while

To avoid the offloading algorithm from getting stuck in high-load calculations of Monte Carlo simulation, we employ artificial neural networks as surrogates to obtain the objective function and constraint satisfaction probability. By leveraging the capabilities of the artificial neural network, we aim to bypass the direct application of Monte Carlo simulation, thereby enhancing computational efficiency without compromising accuracy. The step-by-step procedure encompassing the generation of sample data via Monte Carlo simulation and subsequent training of the artificial neural networks is outlined in Algorithm 7. (1) Monte Carlo simulation is employed to generate training samples. Each training sample is represented as a feature-label pair, where one part comprises the input attributes (features) and the other part comprises the corresponding outputs (labels). The feature vector $(vec(\mathbf{B}), vec(\mathbf{A}), vec(\mathbf{\Lambda}))$ represents the offloading strategy, caching strategy, and CMT data allocation strategy, respectively. The corresponding label vector $\left(F, [Pr_w^{up}]_{w=1}^m, [Pr_w^{down}]_{w=1}^m\right)$ represents the chance-constrained objective value, probabilities of satisfying uplink bandwidth constraints, and probabilities of satisfying downlink bandwidth constraints, respectively. (2) The generated samples are utilized for training two artificial neural networks. A trained artificial neural network is used to calculate the fitness, with the aforementioned three strategies as input and the chance-constrained objective value as output. Another trained artificial neural network is employed to determine whether the chance constraints are satisfied, taking the aforementioned three strategies as input and outputting the probabilities of satisfying

Algorithm 7 The procedure of sample generation and training of artificial neural networks.

Input: task set N, stochastic wireless network distribution
Output: two trained artificial neural networks
for $j = 1$ to n **do**
  $x^O[0][j] \leftarrow rand(0,1)$
  if $x^O[0][j] \geq 0$ **and** $x^O[0][j] < \frac{1}{3}$ **then**
    $x^O[0][j] \leftarrow 0$
  else if $x^O[0][j] \geq \frac{1}{3}$ **and** $x^O[0][j] < \frac{2}{3}$ **then**
    $x^O[0][j] \leftarrow 1$
  else
    $x^O[0][j] \leftarrow 2$
  end if
  $x^O[1][j] \leftarrow rand(0,1)$
  if $x^O[1][j] \geq 0$ **and** $x^O[1][j] < \frac{1}{2}$ **then**
    $x^O[1][j] \leftarrow 0$
  else
    $x^O[1][j] \leftarrow 1$
  end if
  $sum^I[j] \leftarrow 0$
  for $w = 1$ to m **do**
    $x^I[w][j] \leftarrow rand(0, 1 - sum^I[j])$
    $sum^I[j] \leftarrow sum^I[j] + x^I[w][j]$
  end for
end for
repeat
  1–21
until generate a dataset that meets the required quantity
generate stochastic numbers based on the input distribution
calculate the chance constraint value for wireless network capacity constraints
calculate the value of the objective function that satisfies the confidence level l_F
train two artificial neural networks based on the generated data

bandwidth constraints. The generation of training samples via Monte Carlo simulation, followed by the training of the neural network surrogates, is carried out as a preparation phase, entirely preceding the decision-making phase. During the execution of the offloading algorithm, the trained artificial neural networks utilize input parameters including offloading locations, caching conditions, and CMT data allocation conditions as the input to calculate the fitness and to judge whether the chance constraints are satisfied or not. The artificial neural network exhibits the attribute of being trained once and subsequently employed infinitely. Through a single training, neural networks can capture patterns and correlations in data and store them as weight and bias parameters. Upon completion of the training process, the obtained weight and bias parameters can be readily reused as needed, obviating the necessity of retraining the neural network. This advantageous feature enables efficient utilization of the artificial neural network for prediction without incurring the computational overhead associated with repeated training.

5. Experiment and analysis

5.1. Experimental setup

This section presents an extensive series of numerical simulation experiments conducted to validate the established model and evaluate the performance of the proposed offloading algorithm. The experiments were performed on a computer system equipped with an Intel (R) Core(TM) i7-9750H CPU running at a base frequency of 2.6 GHz, 16 GB of RAM, and a 64-bit operating system. The offloading algorithms were implemented using Python 3.8. The artificial neural networks were implemented using PyTorch 2.2 with a fully connected feedforward architecture consisting of 3 hidden layers containing 500, 200, and 50 neurons, respectively. The ReLU activation function was applied after each hidden layer, and the output layer was kept linear. The networks were trained using the Adam optimizer with a learning rate of 0.001 and the mean squared error loss function.

The default simulation parameters are shown in Table 3. The wireless bandwidth fluctuates within its nominal value. We quantify uncertain wireless bandwidth by multiplying the nominal value with a stochastic coefficient. For example, the uplink bandwidth $\widetilde{W_w^{up}}$ of the wth wireless network is expressed as the product of its nominal bandwidth W_w^{up} and stochastic coefficient $\widetilde{\Omega_w}$ ($\widetilde{\Omega_w} \in (0,1]$), that is, $\widetilde{W_w^{up}} = \widetilde{\Omega_w} W_w^{up}$. This experiment makes the assumption that $\widetilde{\Omega_w}$ follows a Beta distribution, denoted as $\widetilde{\Omega_w} \sim Beta(a_w, b_w)$, where a_w and b_w represent the shape parameters associated with the distribution. The shape parameters of the Beta distribution can be flexibly adjusted, enabling the Beta distribution to approximate diverse distributions as the shape parameter values are appropriately selected. For example, setting both shape parameters a_w and b_w to 1 transforms the Beta distribution into a uniform distribution. When the shape parameters a_w and b_w are chosen to be sufficiently large and equal, the Beta distribution closely approximates a normal distribution. Furthermore, the Beta distribution is bounded between 0 and 1, which ensures that the variations in wireless network bandwidth remain within the nominal bandwidth. By modeling the network bandwidth using the Beta distribution, the representation aligns with the inherent constraints of wireless networks, facilitating a more accurate depiction of the actual bandwidth fluctuations experienced in practice.

These parameters are configured based on empirical evidence and generalizability principles. For example, in the real world, the cloud offers the most abundant computational resources, while the edge provides moderate computational resources, and mobile terminals possess the most limited computational resources. Therefore, the computational resources allocated from the cloud (f_i^c) are configured to be twice those of the edge (f_i^e), while the mobile terminal (f^l) is configured with minimal resources. The smartphone, as the most representative mobile terminal, integrates three distinct network interfaces (e.g., 4G, Wi-Fi, and Bluetooth). We configure the number of wireless networks (m) as 3. Parameters configured with interval ranges will randomly generate constant values within their respective bounds. This methodology substitutes fixed-value assignments to improve experimental generalizability. When constructing an application, the attributes of each task are uniformly and randomly generated according to the value ranges specified in Table 3. Prior to generating the dependencies among tasks, the n tasks are randomly permuted to establish a topological order. For any two tasks z_i and z_j, a directed edge from z_i to z_j is considered only if z_i precedes z_j in the topological order. The existence of an edge is determined by a generated random number $\epsilon \in [0,1]$. If $\epsilon \leq \rho$, where ρ is the preset dependency probability, the dependency $\tau_{i,j}$ is set to 1, indicating the presence of an edge; otherwise, $\tau_{i,j} = 0$. Meanwhile, each task has at least one input dependency that originates from its previous task.

5.2. Evaluation of the system model

We first evaluate the impact of system architecture on task offloading performance by comparing the objective values for six different system architectures listed below.

1. End: Tasks are executed on the mobile terminal.
2. Cloud: Tasks are offloaded to the cloud for execution.
3. Edge: Tasks are offloaded to the edge for execution.
4. Cloud-End: The tasks can be executed either locally or offloaded to the cloud for execution.
5. Edge-End: The tasks can be executed either locally or offloaded to the edge for execution.
6. Cloud-Edge-End: The system architecture used in this paper, which facilitates the execution of tasks either locally or through offloading to the edge or cloud for execution.

Table 3
Experimental parameter settings.

Parameter	Definition	Value
f^l	The computational resources of the mobile terminal	7 GHz
f_i^e	The computational resources allocated by the edge for the execution of task z_i	15 GHz
f_i^c	The computational resources allocated by the cloud for the execution of task z_i	30 GHz
H	The privacy risk threshold	1.8
n	The number of tasks	30
m	The number of wireless networks	3
d_i	The input size of task z_i	[500,4000]KB
c_i	The workload of task z_i	[1000,10000]cycles
o_i	The deadline of task z_i	[700,1000]ms
δ_i	The output size of task z_i	[40,100]KB
Δ_i	The cache size of task z_i	[5500]KB
p_i^l	The power of mobile terminal when executing task z_i locally	0.8W
P_0	The idle power of the mobile terminal	0.4W
$P_{i,w}^{up}$	The uplink transmit power of the wth wireless network	[0.3,0.5]W
$P_{i,w}^{down}$	The downlink transmit power of the wth wireless network	[0.1,0.2]W
W_w^{up}	The nominal uplink bandwidth of the wth wireless network	[8,16]MHz
W_w^{down}	The nominal downlink bandwidth of the wth wireless network	[10,20]MHz
a_w	The first shape parameter of the Beta distribution	2
b_w	The second shape parameter of the Beta distribution	8
ξ_1	The first parameter of the utility function	0.002
ξ_2	The second parameter of the utility function	0.5
ξ_3	The third parameter of the utility function	0.001
ξ_4	The fourth parameter of the utility function	0.01
ρ	The dependency probability	0.8

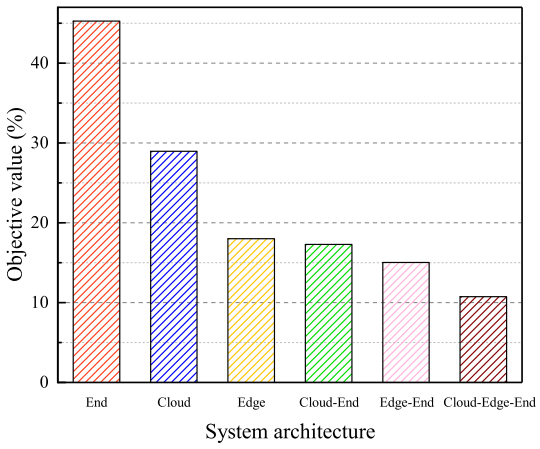


Fig. 5. Objective values under different system architectures.

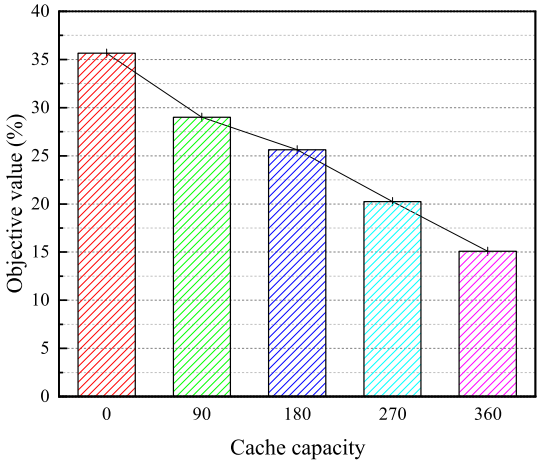


Fig. 6. Objective values under different cache capacities.

As shown in Fig. 5, objective values under different system architectures are compared. Tasks exhibit variations in computational complexity and data transmission volume. Consequently, certain tasks may not be suitable for execution at a specific location due to these differences. Executing tasks at a single location leads to a higher objective value. It is observed that objective values for executing tasks locally, at the cloud, and at the edge are 45.27%, 28.96%, and 18.00%, respectively. As an illustration, it is observed that when employing the "cloud-end" and "edge-end" architectures, the objective values decrease by 11.66% and 2.97% respectively, compared to the scenario where tasks are executed solely at the cloud or edge. The collaborative utilization of cloud, edge, and mobile terminal allows for the execution of tasks at suitable locations, leading to further reductions in the objective value. It is evident that the utilization of "cloud-edge-end" architecture results in a reduction of 6.56% and 4.29% in the objective value when compared to the "cloud-end" and "edge-end" architectures, respectively. Compared to local execution, it can reduce the objective value by 34.53%.

Fig. 6 shows the objective values under different cache capacities. During the course of this experiment, the edge cache capacity is varied from 0 to 360. A cache capacity of 0 signifies the absence of caching capability at the edge, resulting in no tasks being cached. Observations indicate that the objective value is maximized in the absence of edge caching and progressively decreases as the cache capacity increases. Specifically, as the cache capacity increases from 0 to 360, the objective value decreases from 35.64% to 15.09%. The reason behind this phenomenon is that as the cache capacity increases, more tasks are cached at the edge, which in turn prevents more tasks from being fully executed. Consequently, this leads to a reduction in latency and energy consumption, while simultaneously increasing the task utility. However, it is observed that as the cache capacity increases, the rate of decrease in the objective value diminishes. This phenomenon arises due to the interdependencies among different tasks. While being cached at the edge can effectively reduce execution latency and energy consumption for some tasks, it can also introduce additional latency and energy consumption due to data transmission.

Three optimization objectives, namely the latency, energy consumption, and task utility, are taken into consideration in this paper. To consolidate these objectives into a single unified objective, we employed the linear weighting method after performing normalization. The weights assigned to each objective can be adjusted to appropriately emphasize their respective importance in the objective value. The influence of weights on the objective value is shown in Fig. 7. The curves exhibit an ascending trend as the degree of preference for the latency or energy consumption objectives increases. At the same time, some oscillations can be observed within the curves. The underlying reason for this phenomenon is that the optimization objectives prioritize the reduction of latency and energy consumption. Consequently, as the weights assigned to latency or energy consumption increase, the corresponding objective values decrease. However, the weight has a larger impact when multiplied together, leading to an increase in the overall objective values. At times, there may be a larger reduction in latency and energy consumption, resulting in a decrease when multiplied by their weights, which is reflected in the decrease of the objective value. Nevertheless, it can be observed that the curve demonstrates a declining trend with an increase in the degree of preference for the task utility objective. The underlying reason for this phenomenon is that the optimization objective aims to maximize the task utility, as reflected in the minimization of the

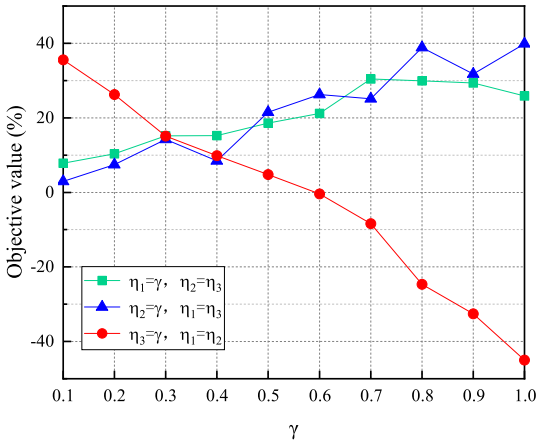


Fig. 7. Objective values under different weights.

objective function by subtracting the product of the utility value and the corresponding weight. Therefore, as the weight increases, the task utility also increases, resulting in a decrease in the objective value when subtracting their product.

Fig. 8 shows objective values under different computing power. As the allocated computational resources at the edge grow from 0.6 times to 1.3 times, the objective value decreases from 22.03% to 13.38%. However, it is worth noting that the curve exhibits an oscillating downward trend. This is mainly because tuning the allocated computational resources at the edge affects the distribution of offloaded tasks on the edge. Under the assumption of fixed edge computational resource capacity, increasing the allocated computational resources for task execution can indeed reduce task consumption. However, it can also impede the offloading of more tasks to the edge. At times, the reduced execution consumption may not sufficiently offset the consumption caused by certain tasks not being offloaded to the edge, resulting in an increase in the objective value. To further validate this explanation, we relaxed the constraint on edge computational resource capacity, treating it similarly to the cloud with infinite computational resources. As anticipated, it can be observed that with the increase in allocated computational resources for task execution, there is a continuous decrease in the objective value. Due to the analogous rationale, the objective value exhibits a continuous decline as the computational resources allocated by the cloud increase. With the increase in allocated computational resources at the cloud from 0.6 times to 1.3 times, there is a corresponding decrease in the objective value, declining from 20.69% to 13.66%.

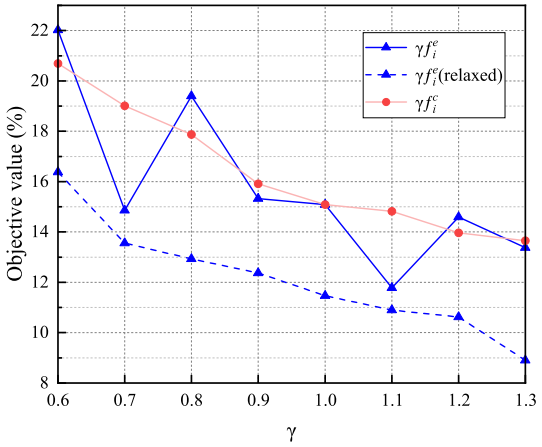


Fig. 8. Objective values under different computing power.

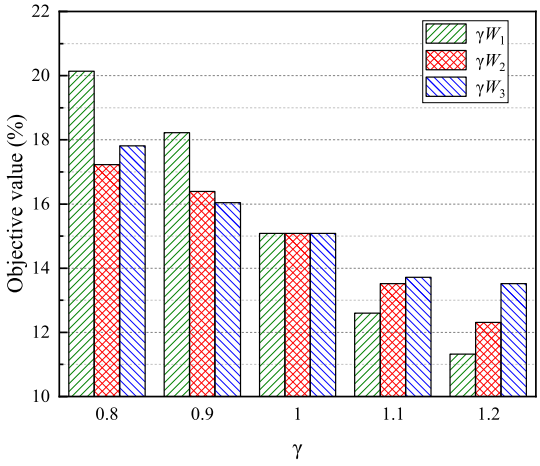


Fig. 9. Objective values under different wireless network bandwidths.

To evaluate the impact of wireless networks on task offloading, we vary the bandwidth of the wireless network to observe the changes in the objective value. Fig. 9 shows objective values under different wireless network bandwidths. In this experiment, three types of wireless networks were configured, and their bandwidths were individually increased from 0.8 times to 1.2 times. It can be observed that as the bandwidth increases, the objective value continuously decreases. Specifically, as the bandwidths of the three wireless networks increase from low to high, the respective objective values decrease from 20.14% to 11.32%, from 17.23% to 12.31%, and from 17.81% to 13.52%. With the continuous increase in wireless network bandwidth, the latency in data transmission significantly decreases, resulting in a corresponding reduction in the objective value. In this experiment, the bandwidth of the three wireless networks decreases sequentially. Therefore, it can also be observed that the magnitude of reduction in the objective value decreases from 8.82% to 4.29%.

5.3. Comparison of the offloading algorithms

To comprehensively evaluate the performance of the proposed offloading algorithm, we select the following five representative comparison algorithms from four categories. TLGWOHW is based on a novel evolutionary algorithm and possesses strong innovation, which is used to verify the competitiveness of the proposed offloading algorithm among innovative algorithms. TLPSO and TLSGA are based on classical evolutionary algorithms and are widely used in the field of task offloading, serving to validate the advancement of the proposed offloading algorithm compared to mainstream algorithms. TLGA is based on the traditional greedy algorithm, and it can verify the global search capability of the proposed offloading algorithm. TLRA is based on the most basic random algorithm and is employed to verify whether the proposed offloading algorithm possesses effective optimization capability. As discussed in Section 4.1.1, interdependencies among the unknown decision variables in the optimization model necessitate decomposing the problem into two sub-problems, thereby enabling a hierarchical solution. Consequently, all comparison algorithms are implemented within this two-layer framework: the outer layer optimizes the offloading and caching decisions, while the inner layer determines the CMT data allocation strategy.

1. HTLIMA: Our proposed hybrid two-layer offloading algorithm is based on the improved genetic algorithm, Monte Carlo simulation, and artificial neural networks.
2. TLGWOHW: The two-layer offloading algorithm is based on the gray wolf optimization algorithm fused with the whale algorithm [51]. TLGWOHW employs a hybrid of gray wolf and whale algorithms to search for optimal strategies in both layers,

integrating the encircling mechanism of the whale algorithm with the gray wolf hierarchy to enhance diversity and convergence. It applies a feasibility rule to eliminate individuals violating chance constraints or other restrictions, steering the population toward feasible regions.

3. TLPSO: The two-layer offloading algorithm is based on the particle swarm optimization algorithm [52]. TLPSO utilizes particle swarm optimization in both layers, leveraging individual memory and social sharing to update velocities and positions toward optima. It adopts the constraint dominance principle to directly discard infeasible particles, retaining only feasible solutions.
4. TLSGA: The two-layer offloading algorithm is based on the standard genetic algorithm [53]. TLSGA adopts the standard genetic algorithm as the search driver within the inner and outer spaces to generate candidate strategies. In contrast to the repair mechanism employed in HTLIMA, it imposes a strict penalty mechanism: any chromosome violating constraints is immediately eliminated, ensuring the feasibility of the final set.
5. TLGA: The two-layer offloading algorithm is based on the greedy algorithm. TLGA adopts a greedy heuristic for decision-making in both layers. Within the feasible solution space, it selects strategies that yield the optimal local objective value at each step based on current information, thereby constructing a feasible solution that satisfies all constraints.
6. TLRA: The two-layer offloading algorithm is based on the random algorithm. TLRA generates candidate strategies through a random walk approach within the feasible region of both layers, without relying on any heuristic information or evolutionary mechanisms.

Fig. 10 shows the comparison of offloading algorithm performance under different numbers of iterations. It is evident that TLRA and TLGA, which do not employ the iterative search mode, exhibit objective values that remain constant over different numbers of iterations, forming a linear relationship. The maximum objective value of TLRA indicates its inferior performance compared to other algorithms. Although TLGA can make decisions greedily, it lacks global search capability, resulting in its objective value being smaller than that of TLRA but still at a relatively high level. It is evident that TLPSO exhibits rapid convergence to the feasible domain during the initial iterations, as compared to TLGWOHW. However, due to its utilization of the discrete particle swarm algorithm in the outer layer and the traditional particle swarm algorithm in the inner layer, the final particle swarm's quality is compromised. Therefore, it can be observed that the performance of TLPSO in later iterations is inferior to that of TLGWOHW. HTLIMA attains the lowest objective value among all compared offloading algorithms, demonstrating superior performance. The underlying rationale behind this observation is that HTLIMA enhances the quality of chromosomes in the population at the initial stage by optimizing the traditional initialization, thereby providing a promising starting point for the search procedure. Throughout the optimization search process, HTLIMA, by incorporating repair-mechanism-based genetic operations, effectively preserves superior genotypic characteristics of good solutions, thereby enhancing search efficiency and reducing the probability of converging to local optima. This explanation can be validated through comparisons with TLSGA. Without implementing the improvements of HTLIMA, TLSGA suffers from infeasible solution interference and premature convergence during iteration. As a result, TLSGA converges to a significantly higher objective value at an accelerated speed compared to HTLIMA.

Fig. 11 shows the comparison of offloading algorithm performance under different confidence levels. It is observed that as the confidence level increases, the objective value also increases. The underlying reason for this phenomenon is that to satisfy the growing confidence level, a larger objective value is required to ensure a sufficient number of stochastic objective values are smaller than it. Although the objective values are increasing, the objective value of HTLIMA remains the smallest, indicating its superior performance compared to other algorithms.

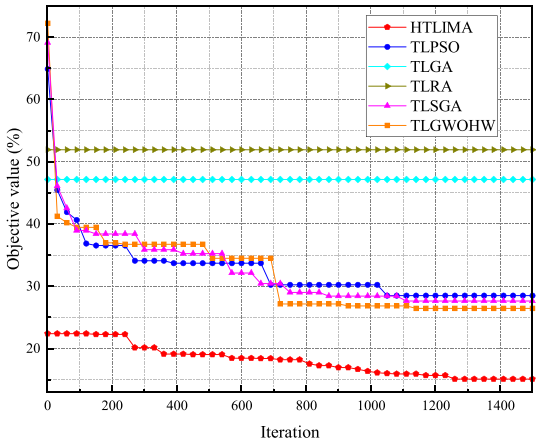


Fig. 10. Comparison of offloading algorithm performance under different numbers of iterations.

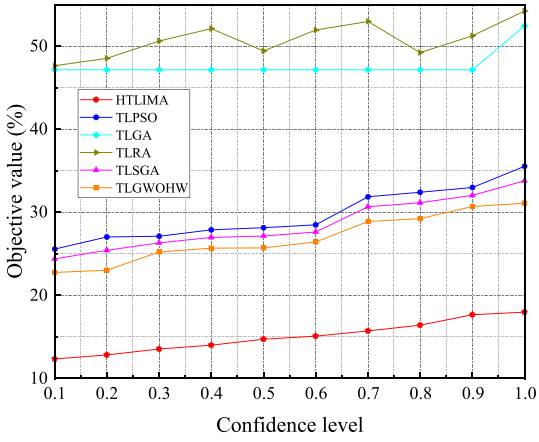


Fig. 11. Comparison of offloading algorithm performance under different confidence levels.

Table 4
Parameters in different scenarios.

Scenario	η_1	η_2	η_3	Δ_i	f^l	f_i^e	f_i^c	W_1^{up}	W_2^{up}	W_3^{up}
S1	0.1	0.3	0.6	0	4.2	18	36	19	13	12
S2	1/3	1/3	1/3	100	5.6	15	30	18	12	16
S3	0.5	0.2	0.3	200	7.7	13.5	27	14	16	12
S4	0.2	0.7	0.1	300	7	12	24	12	11	15
S5	0.1	0.1	0.8	400	8.4	18	36	20	15	13
S6	0.1	0.3	0.6	0	4.2	18	36	20	15	13
S7	1/3	1/3	1/3	100	5.6	15	30	16	12	18
S8	0.1	0.2	0.7	200	7.7	13.5	27	14	16	12
S9	0.2	0.7	0.1	300	9.8	19.5	24	12	11	15
S10	0.1	0.1	0.8	150	8.4	15	36	17	16	15

Moreover, it can be observed that the objective values of TLRA exhibit fluctuations as the confidence level increases. This is because TLRA generates strategies randomly without explicitly aiming to minimize the objective value, resulting in its objective value not necessarily being its minimum achievable value at each confidence level. Therefore, at lower confidence levels, its objective value may be relatively higher, while at higher confidence levels, its objective value may be relatively lower. While the objective value of TLGA is relatively large, it exhibits stability, with significant improvements observed when subjected to high confidence level requirements. This suggests that strategies formulated by TLGA are effective in meeting the demands of confidence level requirements, although they do not guarantee performance.

To comprehensively validate the universal superiority of HTLIMA, we compared it with existing algorithms in different scenarios. The scenario parameters are presented in Table 4, and their units are

Table 5
Objective values (%) of offloading algorithms in different scenarios.

Scenario	HTLIMA	TLPSO	TLGA	TLRA	TLSGA	TLGWOHW
S1	2.39	60.16	55.68	24.50	57.15	40.44
S2	20.18	36.59	38.65	61.21	39.88	31.63
S3	20.63	38.51	78.52	63.00	42.36	60.37
S4	40.20	56.56	44.96	91.46	57.13	51.43
S5	–25.47	–18.07	–14.95	–2.73	–19.52	–20.33
S6	2.96	64.79	10.20	43.87	63.09	49.25
S7	23.54	38.79	41.21	68.46	32.58	33.31
S8	–4.43	7.78	9.22	32.66	7.47	9.81
S9	45.95	62.65	51.75	97.64	53.88	59.06
S10	–18.15	–4.74	10.44	9.04	–3.79	–3.73

consistent with those in Table 3. The default experimental parameters listed in Table 3 are used for the remaining parameters.

The objective values of offloading algorithms in different scenarios are displayed in Table 5, and their performance is visually illustrated through box plots depicted in Fig. 12. The observed results reveal that HTLIMA consistently achieves smaller objective values compared to other algorithms, thereby indicating its superior performance in diverse scenarios. HTLIMA achieves average objective value reductions of 23.52%, 21.79%, 38.13%, 22.24%, and 20.34% compared to TLPSO, TLGA, TLRA, TLSGA, and TLGWOHW, respectively. With enhancements in initialization and genetic operations, HTLIMA demonstrates a heightened ability to effectively explore optimal strategies in comparison to other algorithms. This is the intrinsic driving force behind HTLIMA's ability to obtain the minimum objective value. In S1, the bandwidths of the three wireless networks are decreased compared to those in S6, resulting in higher objective values. In contrast to S2, S7 exhibits a reduction in the bandwidth of the first wireless network while an increase in the bandwidth of the third wireless network. The observed larger objective values in S7 compared to those in S2 demonstrate that the negative impact on the objective value from reducing the first wireless network's bandwidth outweighs the positive effect caused by increasing the third wireless network's bandwidth, indicating the first wireless network holds a greater influence on the objective value. Through the adjustment of weights, S3 exhibits a reduced preference for task utility in comparison to S8. An observation reveals an increase in the objective value in S3 relative to S8. This observation can be attributed to the subtraction employed for task utility within the objective value, whereby an increase in its weight amplifies the reduction effect on the objective value. In comparison to S4, S9 demonstrates improvements in the computing power of the mobile terminal and edge. However, it is noteworthy that the objective value exhibits an increase. This phenomenon further validates the conclusion presented in Fig. 8, which states that increasing the allocation of edge computing power to tasks does not necessarily lead to an improvement in the objective value when the computing capacity of the edge remains fixed. In contrast to S5, S10 exhibits a reduction in cache capacity, accompanied by an increase in the corresponding objective value.

Fig. 13 illustrates the impact of Monte Carlo simulation (MCS) on chance constraint satisfaction. In this experiment, HTLIMA is adopted as the foundational algorithm, and the constraint margin is defined as the difference between the average of $2m$ chance constraint values and the confidence level. It can be observed that the algorithm equipped with MCS achieves strict chance constraint satisfaction across all scenarios, with the objective and constraint margins consistently staying within the positive range. Specifically, the mean margin of the objective function is 0.65%, and that of the constraint is 3.23%. Moreover, the standard deviations of both types of margins are low, at 0.20% and 0.39%, respectively, indicating that the algorithm exhibits good stability in constraint satisfaction. In contrast, the variant without MCS (i.e., trained solely with uniformly sampled data) fails to satisfy the chance constraints in all scenarios, with both objective and constraint margins showing negative values. In terms of distribution characteristics, the deviation below the confidence level is substantial, with mean margins of –38.54% for the objective function and –8.16% for the constraints. Additionally, the standard deviations of these two margins are considerably larger, at 6.92% and 3.89%, respectively, reflecting a strong degree of variability. This suggests that training data lacking probabilistic distribution guidance prevents the artificial neural networks from effectively learning the constraint boundaries, leading to frequent failures in the resulting strategy under uncertainty. Overall, the core contribution of MCS lies in generating training samples according to the probability distributions of the parameters, thereby enabling the artificial neural networks to accurately capture the statistical characteristics of the chance constraints. As a result, the optimization process achieves constraint satisfaction that is predictable, low-variance, and highly reliable. By contrast, without MCS, the training data loses critical probabilistic information, leading to strategies that not only fail to meet constraints but also exhibit high volatility. Therefore, MCS is essential for ensuring feasible decision-making under uncertainty in the proposed algorithm.

Fig. 14 shows the runtime reduction ratio with neural network surrogates. In this experiment, HTLIMA is adopted as the foundational algorithm. The experimental results demonstrate that the neural network surrogates play critical roles in improving HTLIMA's runtime efficiency. Across all scenarios, the runtime reduction ratio achieved by introducing the artificial neural networks remains consistently high, with an average reduction of 87.3%. This result indicates that by replacing the computationally expensive Monte Carlo simulations with artificial neural networks, the substantial computational overhead can

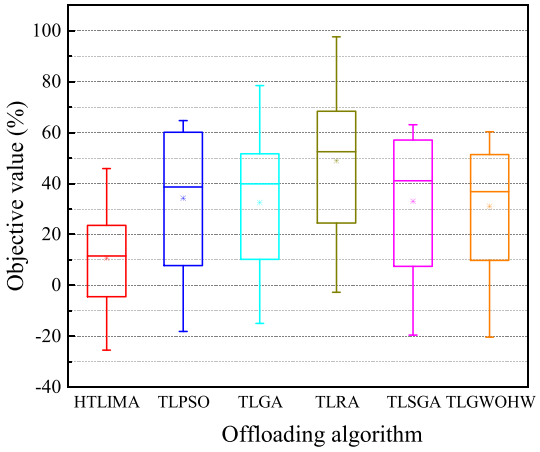


Fig. 12. Objective values of offloading algorithms in different scenarios.

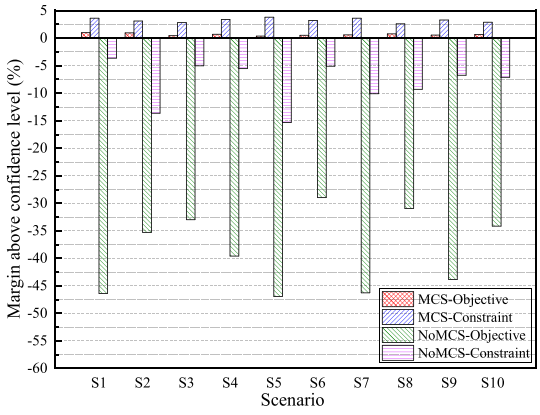


Fig. 13. Impact of Monte Carlo simulation (MCS) on chance constraint satisfaction.

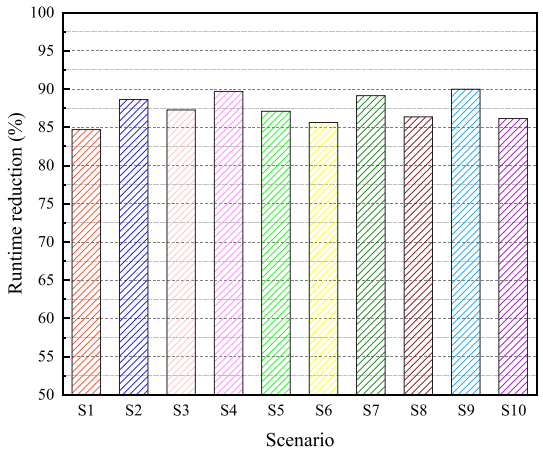


Fig. 14. Runtime reduction ratio with neural network surrogates.

be significantly reduced, thereby greatly enhancing the algorithm's runtime efficiency. Without this surrogate mechanism, the algorithm would have to repeatedly execute a large number of Monte Carlo simulations for each iteration, leading to a sharp increase in runtime and making it difficult to meet the demands of practical applications.

6. Conclusion

This paper aims to address the challenge of the dependent task offloading problem for MEC under uncertainty with a comprehensive consideration of CMT, task caching, and privacy protection. We establish an uncertain joint task offloading optimization model, which employs stochastic numbers to effectively characterize uncertainties and focuses on optimizing the joint strategy to achieve the objectives of reducing latency, decreasing energy consumption, and enhancing task utility. We prove that the established model is NP-hard. To solve the joint strategy encompassing the task offloading strategy, CMT task data allocation strategy, and task caching strategy from the NP-hard model, we propose a hybrid two-layer offloading algorithm that integrates an improved genetic algorithm, Monte Carlo simulation, and artificial neural networks. The established system model and proposed hybrid algorithm offer valuable references and effective methodologies for addressing task offloading problems characterized by coupled uncertainties and task dependencies. The simulation experiments conducted in this paper demonstrate that the evaluation results of the established model align with expectations. The proposed offloading algorithm exhibits superior performance compared to existing algorithms, as evidenced by the experimental findings.

In the future, we will explore the potential for collaborative interactions among multiple edge servers and shift our focus from the single-role decision-making problem to the more complex realm of multi-role decision-making. To address this expanded problem scope, we plan to leverage game theory as a theoretical framework. We aim to establish a novel game-based task offloading model that addresses the complexities of multi-role decision-making in task offloading. We will design innovative distributed algorithms specifically tailored to effectively solve this multi-role decision-making task offloading model. Additionally, we will refine the granularity of privacy risk assessment from task-level to data-level, incorporating an information entropy-based quantification approach (e.g., privacy entropy models).

CRediT authorship contribution statement

Xiaomin Jin: Writing – review & editing, Project administration, Methodology, Conceptualization. **Jingbo Wang:** Writing – original draft, Methodology, Investigation. **Zhongmin Wang:** Writing – review & editing, Visualization. **Yanping Chen:** Writing – review & editing, Visualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was partly supported by the National Natural Science Foundation of China (No. 62301425), the Technology Innovation Leading Program of Shaanxi (No. 2024QY-SZX-17), and the Scientific Research Program Funded by Education Department of Shaanxi Provincial Government (No. 23JP164).

Data availability

Data will be made available upon request.

References

[1] J. Xu, L. Chen, P. Zhou, Joint service caching and task offloading for mobile edge computing in dense networks, in: IEEE 2018 Infocom, 2018, pp. 207–215.
[2] J. Liang, K. Li, C. Liu, K. Li, Joint offloading and scheduling decisions for DAG applications in mobile edge computing, Neurocomputing. 424 (2021) 160–171.
[3] X. Jin, W. Hua, Z. Wang, Y. Chen, A survey of research on computation offloading in mobile cloud computing, wirel, Netw. 28 (2022) 1563–1585.
[4] W.Z. Khan, E. Ahmed, S. Hakak, I. Yaqoob, A. Ahmed, Edge computing: a survey, future gener, Comp. Sy. 97 (2019) 219–235.
[5] Q. Duan, S. Wang, N. Ansari, Convergence of networking and cloud/edge computing: status, challenges, and opportunities, in: IEEE Network, vol. 34, 2020, pp. 148–155.
[6] P. Tomar, G. Kumar, L.P. Verma, V.K. Sharma, D. Kanellopoulos, S.S. Rawat, Y. Alotaibi, CMT-SCTP and MPTCP multipath transport protocols: a comprehensive review, Electronics. 11 (2022) 2384–2421.
[7] A. Islam, A. Debnath, M. Ghose, S. Chakraborty, A survey on task offloading in multi-access edge computing, J. Syst. Architect. 118 (2021) 102225.
[8] M.Y. Akhlaqi, Z.B.M. Hanapi, Task offloading paradigm in mobile edge computing-current issues, adopted approaches, and future directions, J. Netw. Comput. Appl. 212 (2023) 103568.
[9] W. Fan, Y. Su, J. Liu, S. Li, W. Huang, F. Wu, Y. Liu, Joint task offloading and resource allocation for vehicular edge computing based on v2I and V2V modes, in: IEEE T. Intell. Transp, vol. 24, 2023, pp. 4277–4292.
[10] H. Jiang, X. Dai, Z. Xiao, A. Iyengar, Joint task offloading and resource allocation for energy-constrained mobile edge computing, in: IEEE T. Mobile Comput, vol. 22, 2022, pp. 4000–4015.
[11] Y. Gong, K. Bian, F. Hao, Y. Sun, Y. Wu, Dependent tasks offloading in mobile edge computing: a multi-objective evolutionary optimization strategy, future gener, Comp. Sy. 148 (2023) 314–325.
[12] X. Zhang, J. Zhang, Z. Liu, Q. Cui, X. Tao, S. Wang, MDP-based task offloading for vehicular edge computing under certain and uncertain transition probabilities, in: IEEE T. Veh. Technol, vol. 69, 2020, pp. 3296–3309.
[13] Z. Zhou, H. Liao, X. Zhao, B. Ai, M. Guizani, Reliable task offloading for vehicular fog computing under information asymmetry and information uncertainty, in: IEEE T. Veh. Technol, vol. 68, 2019, pp. 8322–8335.
[14] Y. Sun, X. Guo, J. Song, S. Zhou, Z. Jiang, X. Liu, Z. Niu, Adaptive learning-based task offloading for vehicular edge computing systems, IEEE T. Veh. Technol. 68 (2019) 3061–3074.
[15] B. Li, R. Yang, L. Liu, J. Wang, N. Zhang, M. Dong, Robust computation offloading and trajectory optimization for multi-UAV-assisted MEC: a multi-agent DRL approach, in: IEEE Internet Things, vol. 11, 2024, pp. 4775–4786.
[16] X. Zhu, F. Hao, L. Ma, C. Luo, G. Min, L.T. Yang, DRL-based joint optimization of wireless charging and computation offloading for multi-access edge computing, in: IEEE T. Serv. Comput, vol. 18, 2025, pp. 1352–1367.
[17] X. Wang, J. Ye, J. Lui, Decentralized task offloading in edge computing: a multi-user multi-armed bandit approach, in: IEEE 2022 Infocom, 2022, pp. 1199–1208.
[18] J. Wu, Y. Zou, X. Zhang, J. Liu, W. Sun, G. Du, Dependency-aware task offloading strategy via heterogeneous graph neural network and deep reinforcement learning, in: IEEE Internet Things, vol. 12, 2025, pp. 22915–22933.
[19] F. Khoramnejad, A. Syed, W.S. Kennedy, M. Erol-Kantarci, Energy and delay aware general task dependent offloading in UAV-aided smart farms, in: IEEE T. Netw. Serv, vol. 21, 2024, pp. 5033–5048.
[20] S. Li, C. Li, Y. Huang, B.A. Jalaian, Y.T. Hou, W. Lou, Task offloading with uncertain processing cycles, in: 22nd International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing, 2021, pp. 51–60.
[21] W. Chang, Y. Xiao, W. Lou, G. Shou, Offloading decision in edge computing for continuous applications under uncertainty, in: IEEE T. Wirel. Commun, vol. 19, 2020, pp. 6196–6209.
[22] N. Eshraghi, B. Liang, Joint offloading decision and resource allocation with uncertain task computing requirement, in: IEEE 2019 Infocom, 2019, pp. 1414–1422.
[23] K. Peng, J. Nie, K. Neeraj, C. Cai, J. Kang, Z. Xiong, Y. Zhang, Joint optimization of service chain caching and task offloading in mobile edge computing, Appl. Soft Comput. 103 (2021) 107142–110726.

[24] X. Liu, X. Zhao, G. Liu, F. Huang, T. Huang, Y. Wu, Collaborative task offloading and service caching strategy for mobile edge computing, Sensors. 22 (2022) 6760.
[25] Q. Shen, B. Hu, E. Xia, Dependency-aware task offloading and service caching in vehicular edge computing, in: IEEE T. Veh. Technol, vol. 71, 2022, pp. 13182–13197.
[26] H. Feng, S. Guo, L. Yang, Y. Yang, Collaborative data caching and computation offloading for multi-service mobile edge computing, in: IEEE T. Veh. Technol, vol. 70, 2021, pp. 9408–9422.
[27] Z. Xiao, J. Shu, H. Jiang, J.C.S. Lui, G. Min, J. Liu, S. Dustdar, Multi-objective parallel task offloading and content caching in D2D-aided MEC networks, in: IEEE T. Mobile Comput, vol. 22, 2022, pp. 6599–6615.
[28] Z. Ma, S. Zhang, N. Chen, Z. Qian, Q. Gu, Y. Liang, S. Lu, Dependent task offloading and service caching with state management for mobile edge computing, in: IEEE 2023 International Conference on Communications, 2023, pp. 6249–6254.
[29] J. Zhang, G. Zhang, X. Bao, C. Ding, P. Yuan, X. Zhang, S. Wang, Dependent application offloading in edge computing, in: IEEE T. Cloud Comput, vol. 11, 2023, pp. 3439–3451.
[30] L. Zhao, Z. Zhao, A. Hawbani, Z. Liu, Z. Tan, K. Yu, Dynamic caching dependency-aware task offloading in mobile edge computing, in: IEEE T. Comput, vol. 74, 2025, pp. 1510–1523.
[31] Y. Zhang, Q. Yu, Joint optimization of edge hybrid caching and task offloading for bi-directional computing tasks in wireless edge networks, in: 10th International Conference on Intelligent Computing and Signal Processing, 2025, pp. 673–678.
[32] Z. Song, Y. Liu, X. Sun, Joint task offloading and resource allocation for NOMA-enabled multi-access mobile edge computing, IEEE T. Commun. 69 (2022) 1548–1564.
[33] X. Jin, Y. Liu, W. Fan, W. Fan, B. Tang, Multisite computation offloading in dynamic mobile cloud environments, Sci. China Inform. Sci. 60 (2017) 089301–089303.
[34] F. Wang, J. Xu, Z. Ding, Multi-antenna NOMA for computation offloading in multiuser mobile edge computing systems, in: IEEE T. Commun, vol. 67, 2018, pp. 2450–2463.
[35] M. Li, S. Yang, Z. Zhang, J. Ren, G. Yu, Joint subcarrier and power allocation for OFDMA based mobile edge computing system, in: IEEE 28th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications, 2017, pp. 1–6.
[36] S. Nath, Y. Li, J. Wu, P. Fan, Multi-user multi-channel computation offloading and resource allocation for mobile edge computing, in: IEEE 2020 International Conference on Communications, 2020, pp. 1–6.
[37] Z. Cao, P. Zhou, R. Li, S. Huang, D. Wu, Multiagent deep reinforcement learning for joint multichannel access and task offloading of mobile-edge computing in Industry 4.0, in: IEEE Internet Things, vol. 7, 2020, pp. 6201–6213.
[38] J. Xue, Y. An, Joint task offloading and resource allocation for multi-task multi-server NOMA-MEC networks, IEEE Access. 9 (2021) 16152–16163.
[39] Y. Yu, J. Zhang, K.B. Letaief, Joint subcarrier and CPU time allocation for mobile edge computing, in: IEEE 2016 GLOBECOM, 2016, pp. 1–6.
[40] Z. Wang, Y. Ding, X. Jin, Y. Chen, C. Gao task offloading for edge computing in industrial internet with joint data compression and security protection, J. Supercomput. 79 (2023) 4291–4317.
[41] A. Samy, A. Ibrahim, H. Yu, W. Zhang, H. Zhang, Secure task offloading in blockchain-enabled mobile edge computing with deep reinforcement learning, in: IEEE T. Netw. Serv, vol. 19, 2022, pp. 4872–4887.
[42] S. Luo, H. Li, Z. Wen, B. Qian, G. Morgan, A. Longo, O. Rana, R. Ranjan, Blockchain-based task offloading in drone-aided mobile edge computing, in: IEEE Network, vol. 35, 2021, pp. 124–129.
[43] Y. He, D. Zhai, F. Huang, D. Wang, X. Tang, R. Zhang, Joint task offloading, resource allocation, and security assurance for mobile edge computing-enabled UAV-assisted VANETs, Remote Sens. 13 (2021) 1547–1572.
[44] X. Xu, C. He, Z. Xu, L. Qi, S. Wan, M.Z.A. Bhuiyan, Joint optimization of offloading utility and privacy for edge computing enabled IoT, in: IEEE Internet Things, vol. 7, 2019, pp. 2622–2629.
[45] Y. Sun, N. Li, X. Tao, Privacy preserved secure offloading in the multi-access edge computing network, in: IEEE 2021 Wireless Communications and Networking Conference Workshops, 2021, pp. 1–6.
[46] J. Fan, J. Wu, S. Mumtaz, Collaborative-filtering privacy-preserving vehicular edge computation offloading in green smart cities, in: IEEE 2023 International Conference on Communications, 2023, pp. 2455–2460.
[47] X. Zhu, F. Hao, M. Lei, A. Nasridinov, J. Shang, Z. Yu, L. Guo, Trust-aware caching-constrained tasks offloading in multi-access edge computing, future gener, Comp. Sy. 175 (2026) 108033.
[48] D. Huang, P. Wang, D. Niyato, A dynamic offloading algorithm for mobile computing, in: IEEE T. Wirel. Commun, vol. 11, 2012, pp. 1991–1995.
[49] G. Baranwal, D.P. Vidyarthi, Computation offloading model for smart factory, J. Amb. Intel. Hum. Comp. 12 (2021) 8305–8318.
[50] M.R. Garey, D.S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, W.H. Freeman and Co. Press, San Francisco, USA, 1979.
[51] W. Zhang, K. Tuo, Research on offloading strategy for mobile edge computing based on improved grey wolf optimization algorithm, Electronics. 12 (2023) 2533.
[52] N. Bacanin, M. Antonijevic, T. Bezdan, M. Zivkovic, K. Venkatachalam, S. Malebary, Energy efficient offloading mechanism using particle swarm optimization in 5G enabled edge nodes, Cluster Comput. 26 (2023) 587–598.
[53] S. Chakraborty, K. Mazumdar, Sustainable task offloading decision using genetic algorithm in sensor mobile edge computing, J. King Saud Univ.-Com. 34 (2022) 1552–1568.